



Xi'an Jiaotong-Liverpool University
西交利物浦大學

SCHOOL OF ADVANCED TECHNOLOGY

SAT301 FINAL YEAR PROJECT

*Direct Optimal Action Learning: A Versatile
Framework for Q-Value Based Policy Extraction*

Final Thesis

In Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Science

Student Name	:	Xu Chen
Student ID	:	2257453
Supervisor	:	Chunchuan Lyu
Assessor	:	

Abstract

Training expressive policies, such as Diffusion and Flow models, via behavior-regularized objectives is often computationally prohibitive. We propose Direct Optimal Action Learning (DOAL), a framework that leverages native behavior losses (e.g., flow matching) to replicate “optimal” actions derived from Q-values. A core strength of DOAL is its broad applicability across various architectures and value functions; we demonstrate its effectiveness by integrating it with Gaussian, Diffusion, and Flow policies, as well as both standard and regularized Q-value estimation methods. Furthermore, we introduce a Batch-Normalizing Optimizer by reinterpreting the Q-loss balancing factor as a trust-region mechanism. This innovation offers a unified optimization strategy that remains stable across various policy distributions and value function types, thereby significantly reducing the need for policy-specific hyperparameter tuning. Evaluated on OGBench and D4RL Adroit, DOAL consistently outperforms strong baselines. We further evaluated DOAL on Brax environments under online settings. The code is available through [Github Code](#).

Keywords: Reinforcement learning; Offline RL; Behavior cloning; Flow matching; Policy extraction

Acknowledgements

First and foremost, I would like to express my deepest and most sincere gratitude to my supervisor, Professor Chunchuan Lyu. Words cannot fully capture how grateful I am for his hands-on guidance and unwavering patience throughout my undergraduate research. He did not merely supervise my work from a distance; instead, he taught me step-by-step, guiding me through complex mathematical derivations and refining my research intuition. His scholarly rigor, dedication, and kindness have set a profound example for me, not only as an aspiring researcher but also as a person.

I am infinitely grateful to my parents for their unconditional love and lifelong support. Choosing Computer Science as my major and dedicating myself to the demanding field of Machine Learning was not an easy path, yet they trusted my vision completely. Their continuous encouragement has been the bedrock of my academic pursuit. I also owe a huge debt of gratitude to my dear friends. During the lowest ebbs of my life, their constant companionship, understanding, and emotional support kept me grounded and resilient. Empowered by this collective trust, understanding, and support, I am incredibly excited to embark on the next chapter of my academic journey at the University of Washington (UW) for the MS ECE program.

As an undergraduate, the more I delved into the top-tier AI area, the more I realized that the works which truly stand the test of time and leave a permanent mark on history are never purely empirical applications. From architectural milestones like LeNet, ResNet, and CNNs, to generative paradigms like DDPM, Flow Matching, Transformers, Mamba, and KAN, and further to RL foundations like DQN, PPO, and SAC—all of these monumental breakthroughs demand extraordinary mathematical rigor and theoretical depth. I strongly believe that during the undergraduate stage, one should focus on foundational, high-quality research rather than superficial engineering.

Ultimately, I chose to dedicate myself to Reinforcement Learning because I believe it transcends mere engineering—it is deeply intertwined with philosophy. In our daily lives, we constantly contemplate the meaning of being human, which mirrors how we define an intelligent agent, formulate its exploration strategies, and design its reward mechanisms.

Contents

Abstract	ii
Acknowledgements	iii
List of Tables	vii
List of Figures	viii
List of Acronyms	ix
1 Introduction	1
1.1 Motivation	1
1.2 Research Aims and Objectives	2
1.3 Thesis Contributions	2
1.3.1 A Novel Policy Extraction Paradigm	2
1.3.2 A Batch-Normalizing Trust Region Optimizer	3
1.3.3 Decoupled Investigation of Value and Policy Backbones:	3
1.3.4 Comprehensive Empirical Validation	3
1.4 Literature Review	3
1.4.1 Regularized Value Estimation	4
1.4.2 Policy Extraction for Diffusion Models	4
1.4.3 Policy Distillation with Regularized Actor-Critic.	5
1.4.4 Long Horizon Problems	5
1.4.5 Non End2End Optimization	5
2 Background	6
2.1 Foundations of Deep Q-Networks (DQN)	6
2.1.1 Mathematical Formulation and Bellman Optimality	6
2.1.2 Loss Function and Gradient Descent	7
2.2 The Distributional Shift Barrier in Offline Contexts.	7
2.3 Flow Matching	7
2.4 TrigFlow	8
2.5 Max Q Sampling	8
2.6 Behavior-Regularized Actor-Critic (BRAC)	9
3 Methodology	10
3.1 The Optimal Action	10

3.1.1	Intuitive Perspective.	10
3.1.2	Formal perspective.	11
3.2	Batch-Normalizing Optimizer	12
3.3	The Batch-Normalized DOAL Objectives	13
4	Experiments	14
4.1	Toy Experiment Results	14
4.2	Main Experiments Setup	14
4.3	Evaluation and Benchmarks	17
4.4	Main Results	17
4.5	Discussion	19
4.5.1	Importance of Regularization	19
4.5.2	Importance of Tanh	19
4.5.3	Time Complexity	19
4.5.4	Choose Statistical Trust Region δ	20
5	Conclusion and Future Work	22
5.1	Conclusion	22
5.2	Future Work	22
5.2.1	Algorithmic Extension to Online Adaptation	23
5.2.2	Uncertainty-Aware N -Ensemble Mechanics	23
	References	25
A	Proof of Proposition 3.1	32
B	Proof of Proposition 3.3	33
C	MaxQ Sampling needs Balancing	35
D	Value Functions	37
E	DOAL Objectives	38
E.1	Directed Implicit Q-Learning (DIQL)	38
E.2	Direct Implicit Flow-Q-Learning (DIFQL)	38
E.3	Efficient Implicit TrigFlow Q-Learning (ETrigFlow)	39
E.4	Directed Implicit TrigFlow Q-Learning (DTrigFlow)	39
E.5	Directed MultiStep Flow Q-Learning (DMFQL)	39
E.6	Directed MultiStep Flow Regularized Q Learning (DMFReBRAC)	40
F	Experiment Results	41

G	Ablation Study	42
G.1	Choosing n_{sample}	42
G.2	Choosing δ	43
G.3	Importance of Behavior Clone Coefficient α	43
G.4	Choosing $n_{\text{target_sample}}$	44
H	Hyperparameters	46

List of Tables

4.1	IQL-based offline RL results. Full results on the OGBench and D4RL tasks with IQL value function. They are the default single-task in each environment.	17
4.2	Q-Learning based offline RL results. The results are averaged over 8 seeds . The ReBRAC*, FQL* results are collected from Flow Q learning [1].	18
4.3	Different Environments with the Observed Mean $ \nabla_a Q(s, a) $, α , and δ	20
G.1	Comparison of MFQL Performance Across Different n_{sample} averaging over 4 seeds	42
H.1	DOAL Related Hyperparameters	46
H.2	Other Hyperparameters	46

List of Figures

3.1	DOAL Framework. [LEFT] End2end re-parameterized policy gradient from the Q and the behavior clone loss. [RIGHT] DOAL framework, where we extract an optimized a^{target} from Q, and use policy dependent behavior clone loss.	10
4.1	Performance ratio under unchanged conditions, where $n=5$	15
4.2	Performance ratio under unchanged conditions, where $n=10$	15
4.3	Performance ratio under unchanged conditions, where $n=50$	15
4.4	Performance ratio under unchanged conditions, where $n=100$	16
4.5	Performance ratio under unchanged conditions, where $n=500$	16
4.6	Runtime and Computational Complexity . A regression line models the actual running time by the number of function calls. The actual run time is on the antmaze-large with an A800.	19
4.7	Mean Batch Normalized Gradient Norm of DMFQL across OGbench	21
4.8	The relationship between MFQL, DMFQL, MReBRAC and DMReBRAC.	21
5.1	Performance of DMFQL and TD3 on the Brax Environments in 1 seed. . . .	23
F.1	Ogbench several environment visualizations	41
F.2	Performance Comparison of Algorithms on the OGbench Environment. . . .	41
G.1	Performance Comparison of DMFQL on the OGbench Environment with $\delta = 0.03, 0.1, 0.3$ over 4 seeds.	43
G.2	Performance Comparison of DMFQL on the OGbench Environment with α^* from FQL and $\alpha = 1$ over 4 seeds.	44
G.3	Performance Comparison of DMFQL on the OGbench Environment with target n sample = 4 and target n sample = 1 over 4 seeds.	45

List of Acronyms

Term	Initial components of the term
DRL	Deep Reinforcement Learning
Offline RL	Offline Reinforcement Learning
MDP	Markov Decision Process
MLP	Multi-Layer Perceptron
RL	Reinforcement Learning
TD3	Twin Delayed Deep Deterministic policy gradient algorithm
BRAC	Behavior Regularized Actor-Critic
DOAL	Direct Optimal Action Learning
AWR	Advantage Weighted Regression
DIQL	Direct Implicit Q Learning
DIFQL	Direct Implicit Flow Q Learning
DTrigflow	Direct TrigFlow
FQL	Flow Q Learning
IFQL	Implicit Flow Q Learning
IQL	Implicit Q Learning
ReBRAC	Revisited Behavior Regularized Actor-Critic
DMFQL	Direct MultiStep Flow Q Learning
MFQL	MultiStep Flow Q Learning
DMFReBRAC	Direct MultiStep Flow Revisited Behavior Regularized Actor-Critic
MReBRAC	MultiStep Flow Revisited Behavior Regularized Actor-Critic

Chapter 1 Introduction

1.1 Motivation

Reinforcement learning (RL) has emerged as a powerful paradigm for autonomous decision-making, demonstrating remarkable success in complex environments ranging from strategic games to simulated robotic control [1–3]. Typically, standard online RL framework relies on an active trial-and-error process, where the agent continuously interacts with the environment to collect fresh transitions and refine its policy through immediate feedback. However, this unconstrained exploratory nature severely impedes its deployment in real-world applications, such as autonomous driving, clinical healthcare, and industrial automation, where active exploration is either prohibitively expensive, logistically impractical, or fundamentally unsafe [2, 3]. Consequently, there is a compelling need to shift the paradigm from active online exploration to exploiting existing, static datasets collected by historical behaviors. This pragmatic necessity directly motivates the field of offline reinforcement learning.

Offline reinforcement learning aims at efficiently and effectively extracting policy from experience beyond the simple imitation learning [2, 3]. For offline RL agents to perform better than a simple behavior cloning agent, value estimation and extracting information from the estimated value function are the key [4]. Yet, due to the lack of interaction, agents cannot extrapolate too much to avoid the distribution shift. Hence, the success of offline RL depends on balancing the maximization of the estimated Q value and behavior cloning [5–8].

Meanwhile, as the scale and diversity of offline datasets continue to grow, there is an increasing need for policies capable of modeling highly multi-modal and complex action distributions. Diffusion and flow matching models [9–14] have recently been applied to Offline RL [1, 15–18], and improved the modeling capacity of policy distribution.

However, efficient policy extraction requires a reparameterized policy gradient through the Q value function [4], and it is non-trivial for distribution involving an iterative sampling procedural [1, 8, 18, 19]. Existing solutions usually involve some computation overhead, require careful tuning of hyperparameters across environments, or are not as effective [16, 18, 20, 21].

1.2 Research Aims and Objectives

The overarching **aim** of this dissertation is to develop a data-driven, offline reinforcement learning (RL) framework capable of training high-capacity policies without environmental interaction. Specifically, this research focuses on simultaneously overcoming two core vulnerabilities in current offline workflows: mitigating the severe catastrophic value overestimation caused by out-of-distribution (*OOD*) **distributional shifts**, and eliminating the prohibitive computational overheads incurred by **Backpropagation Through Time (BPTT)** or iterative sampling loops during the policy extraction phase.

To achieve this primary research aim, the study is structured around the following concrete, sequentially dependent **research objectives**:

1. To formulate a novel policy distillation paradigm that bypasses traditional end-to-end backpropagation through complex iterative generative models (e.g., Diffusion or Flow Matching policies). The target mechanism must eliminate the reliance on massive runtime sampling loops during training, shifting the heavy computational burden away from the policy’s structural parameterization to accelerate convergence.
2. To design a robust, localized regularization mechanism within the value-directed action optimization phase. This mechanism must mathematically safeguard value estimation by constructing a scalable gradient-based trust region, ensuring that optimized target actions are effectively constrained away from unreliable OOD regions without relying on brittle, environment-specific global hyperparameter tuning.
3. To theoretically adapt and evaluate the proposed decoupled optimization mechanism across a multi-variant matrix of offline RL foundations. This involves expanding the mechanism’s compatibility across various value backbones (such as In-Sample Learning and Regularized Q-Learning) and modern expressive policy parameterizations (including Gaussian baselines, Flow Matching, and Diffusion models).

1.3 Thesis Contributions

1.3.1 A Novel Policy Extraction Paradigm

As illustrated in Figure 3.1, we deviate from conventional end-to-end backpropagation through the Q-value function. Instead, DOAL decouples the optimization into two efficient stages:

generating an optimized target action a^{target} via the Q-value function and the behavior regularization, followed by updating the policy network using its native, highly efficient loss function (e.g., velocity matching loss for Flow models).

1.3.2 A Batch-Normalizing Trust Region Optimizer

To safeguard policy extraction against unreliable, uncalibrated out-of-distribution (OOD) Q-value estimations, we propose a batch-normalized action-update vector:

$$g(s, a) = \frac{\delta}{\mathbb{E}_{(s', a') \sim \mathcal{B}} [\|\nabla_{a'} Q_\phi(s', a')\|_2]} \cdot \nabla_a Q_\phi(s, a) \quad (1.1)$$

This formulation bounds the action shift within a localized trust region δ , ensuring that the optimization step size remains stable and proportional to the local value gradient.

1.3.3 Decoupled Investigation of Value and Policy Backbones:

By leveraging Implicit Q-Learning (IQL), we successfully isolate value estimation from the policy extraction loop. Furthermore, we identify that the action candidate sample size (n_{sample}) in MaxQ sampling is a critical, previously neglected hyperparameter that governs the fundamental trade-off between maximization bias and dataset coverage support.

1.3.4 Comprehensive Empirical Validation

We conduct extensive evaluations across 9 default tasks on OGBench and 6 Adroit tasks on D4RL datasets, encompassing three classes of value functions and three policy architectures. The results demonstrate that DOAL consistently outperforms strong baselines, including the previously state-of-the-art FQL [1], while introducing minimal computational overhead (only one extra forward and backward call) and maintaining high hyperparameter sharing robustness.

1.4 Literature Review

To contextualize the scientific boundaries of this dissertation and highlight the core theoretical deficiencies within current paradigms, this section provides a structured overview of the existing literature intersecting with our research objectives. The development of a sample-efficient, expressive, and robust offline policy extraction framework requires an integration

of insights from multiple computational subfields. Specifically, we review the foundational methodologies in regularized value estimation that attempt to suppress distribution shifts, examine contemporary strategies for policy extraction within high-capacity generative models (such as Diffusion and Flow Matching models), analyze the intersections of regularized actor-critic systems, and bridge our work with broader orthogonal domains including long-horizon trajectory reduction and non-end-to-end neural network optimization paradigms.

1.4.1 Regularized Value Estimation

In-Sample Optimization (e.g., IQL [7]; X-QL [22]; SQL and EQL [23]) addresses offline RL by performing in-sample value iteration, avoiding queries to out-of-distribution (OOD) actions and directly approximating the optimal value function; **Conservative Methods** (e.g., CQL [24]; EDAC [25]; AWAC [26]; BCQ [27]; ReBRAC [8]) propose methods that penalize out-of-distribution actions.

1.4.2 Policy Extraction for Diffusion Models

Accelerated Sampling Techniques. A prominent research direction focuses on accelerating the sampling process of pre-trained generative policies. EDP [18] reformulates the reverse denoising process to estimate the target action a_0 in a single step. While these approaches achieve notable improvements in sampling efficiency, they remain inherently heuristic. Meanwhile, FQL [1] circumvents the sampling via a student network with one-step sampling.

Value Guidance Methods. A substantial body of work in offline RL leverages diffusion models to approximate the behavior policy underlying the dataset. One prevalent strategy involves using the gradient of Q-value functions to guide the action generation process: QGPO [16], SFBC [20], EDA [28], QVPO [29], and CFGRL [30]. An alternative, more straightforward approach modulates the policy by re-weighting transition samples based on their estimated values [30, 31].

MaxQ Sampling. MaxQ sampling has been introduced in early work with diffusion models. However, they mostly set n_{sample} to be a large number and thought the trade-off is between computation budget vs. quality [1, 16, 18]. Some other work used weighted resampling instead of MaxQ sampling [17], where proper weighting can alleviate the overestimation bias.

As for MaxQ sampling, [32] suggested larger n_{sample} is better. A very recent on-horizon reduction [33] tuned the n_{sample} , arguing that for large n_{sample} , the sampled distribution might deviate from the policy distribution too much. Yet, they did not discuss the over-estimation bias of the Q value (see Appendix C).

1.4.3 Policy Distillation with Regularized Actor-Critic.

Regularized value function learning has been shown to be effective [17, 25]. Contemporary methods like DAC [34] and BDPO [35] further integrate diffusion models with the regularized actor-critic framework. In particular, a very recent work, FAC [36], builds on FQL and add carefully designed regularization on the Q value function, achieved very strong performance.

1.4.4 Long Horizon Problems

For challenging tasks that most algorithm cannot address, like antmaze-giant-navigate, [33, 37, 38] looked at the horizon reduction issue. While this is orthogonal to our problem, such methods might be necessary for challenging environments.

1.4.5 Non End2End Optimization

Similar to our DOAL at a conceptual level, there are several alternatives to end2end back-propagation that have been explored for neural network training. **Structured Reinforcement Learning** [39] essentially develops the same idea as ours for combinatorial actions with Fenchel-Young losses [40]. **Target-Propagation.** [41, 42] propose to compute targets rather than gradients, at each layer. **Nested-Learning.** [43] trains neural networks with nested component-wise associative memory learning problems. It is not hard to see that their formulation can be converted to target matching like DOAL. **Nonparametric Learning.** [44] provably learns a two-layer network by matching the first layer output with the ideal second layer input.

The remainder of this dissertation is organized as follows. Chapter 2 provides background on the offline reinforcement learning and generative models. Chapter 3 formulates the DOAL methodology. Chapter 4 presents the experimental setup, results, and analysis. Finally, Chapter 5 concludes the work and outlines future directions.

Chapter 2 Background

We consider a Markov decision process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, r, p, \gamma, \rho_0)$ [45], where $\mathcal{S}$ is the state space, $\mathcal{A} = \mathbb{R}^d$ is a d -dimensional continuous action space, $r(s, a)$ is the reward function, $p(s' \mid s, a)$ is the transition dynamics, $\gamma \in [0, 1)$ is the discount factor, and $\rho_0(s)$ is the initial state distribution. In offline RL, the agent learns from a fixed dataset $\mathcal{D} = \{(s_i, a_i, s'_i, r_i)\}_{i=1}^N$ consisting of individual transitions rather than complete trajectories. The objective is to learn a policy $\pi_\theta(a \mid s)$ that maximizes the expected discounted return $G(\pi_\theta) = \mathbb{E}_{\pi_\theta} [\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$, while avoiding distributional shift caused by querying actions outside the support of $\mathcal{D}$.

2.1 Foundations of Deep Q-Networks (DQN)

Before delving into advanced offline policy extraction frameworks, it is crucial to review the baseline mechanics of the Deep Q-Network (DQN) [46]. DQN extends classical tabular Q-learning to high-dimensional continuous state spaces by utilizing a deep neural network to approximate the optimal action-value function, bypassing the curse of dimensionality.

2.1.1 Mathematical Formulation and Bellman Optimality

The core objective of DQN is to learn a parametric function $Q_\theta(s, a) \approx Q^*(s, a)$, which predicts the expected cumulative discounted return of taking action a in state s and subsequently following the optimal policy. This formulation is anchored on the **Bellman Optimality Equation**:

$$Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p} \left[\max_{a' \in \mathcal{A}} Q^*(s', a') \right] \quad (2.1)$$

where $r(s, a)$ denotes the immediate reward, $p(s' \mid s, a)$ represents the transition dynamics, and $\gamma \in [0, 1)$ is the discount factor. In a deep learning framework, this identity is transformed into a regression problem where the right-hand side serves as the dynamic regression target.

2.1.2 Loss Function and Gradient Descent

To optimize the network parameters θ , DQN minimizes the Mean-Squared Bellman Error (MSBE) over transitions sampled from the environment. The loss function at iteration t is defined as:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}_{\text{replay}}} \left[\left(r + \gamma \max_{a' \in \mathcal{A}} Q_{\theta^-}(s', a') - Q_{\theta}(s, a) \right)^2 \right] \quad (2.2)$$

Here, two distinct sets of parameters are maintained to decouple the learning dynamics:

- θ : The weights of the **prediction network** (Current Q-Network) being actively optimized via backpropagation.
- θ^- : The weights of the **target network**, which are periodically frozen or smoothly tracked to compute stable target values.

In practice, optimization is executed over a mini-batch of transitions $\mathcal{B}$ sampled uniformly from the replay buffer, updating the parameters via stochastic gradient descent (SGD) [47].

2.2 The Distributional Shift Barrier in Offline Contexts.

While the maximum operator $\max_{a' \in \mathcal{A}} Q_{\theta^-}(s', a')$ works effectively in online settings, it becomes the primary failure point when transitioning to **offline reinforcement learning**. In an online setting, overestimation errors resulting from querying an unobserved or sub-optimal action a' can be actively corrected when the agent executes a' in the real environment and receives factual environmental feedback.

However, in an offline setting, the agent is restricted to a fixed dataset $\mathcal{D}$ without environment interaction. Consequently, the maximization step frequently queries Out-of-Distribution (OOD) actions where the Q-network has poorly generalized. This unconstrained maximization causes function overestimation errors to accumulate exponentially along the Bellman backup chain, resulting in a total collapse of policy extraction—a challenge that modern offline algorithms explicitly seek to regularize.

2.3 Flow Matching

Flow Matching (FM) establishes a deterministic path $(p_t)_{t \in [0,1]}$ that continuously transforms a source distribution p_1 into the target behavior distribution p_0 , with each p_t defined over

$\mathbb{R}^d$ [12, 48]. We adopt the simplest instantiation of FM, utilizing linear interpolation paths with uniform time sampling [1, 48]. For an action $a_0 \sim \pi(a_0)$, we learn a time-dependent velocity field $v_\theta(a, t) : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ through the following loss:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{a_0 \sim q(a_0), a_1 \sim \mathcal{N}(0, I)} [\|v_\theta(a_t, t) - (a_0 - a_1)\|^2], \quad a_t = (1 - t)a_0 + ta_1. \quad (2.3)$$

This formulation ensures training stability while admitting efficient sampling through explicit Euler discretization of the underlying flow ODE:

$$a_{t-\Delta t} = a_t + \Delta t \cdot v_\theta(a_t, t), \quad \Delta t = \frac{1}{N} \quad (2.4)$$

where N represents the number of flow steps.

2.4 TrigFlow

We adapt the TrigFlow [49] to train diffusion policy [9–11], defining the forward diffusion process as follows: given $a_0 \sim p(a_0)$ and $z \sim \mathcal{N}(0, I)$, the noisy sample at time t is given by

$$a_t = \cos(t)a_0 + \sin(t)z, \quad t \in [0, \frac{\pi}{2}] \quad (2.5)$$

with $a_{\pi/2} \sim \mathcal{N}(0, I)$. The model can be trained to minimize the prediction error:

$$\begin{aligned} \mathcal{L}_{\text{diff}}(\theta) &= \mathbb{E}_{a_0 \sim p(a_0), z \sim \mathcal{N}(0, I), t \sim \mathcal{U}[0, \pi/2]} [\|f_\theta(a_t, t) - a_0\|_2^2], \\ f_\theta(a_t, t) &= \cos(t)a_t - \sin(t) \cdot F_\theta(a_t, t), \end{aligned} \quad (2.6)$$

where F_θ is a learned network and f_θ naturally satisfies the boundary condition $f_\theta(a_0, 0) = a_0$.

We implement a first-order DDIM [50] sampling scheme for efficient inference:

$$a_t = \cos(k - t) \cdot a_k - \sin(k - t) \cdot F_\theta(a_k, k), \quad (2.7)$$

where k is the previous time step. In our paper, we divide the steps evenly into 10.

2.5 Max Q Sampling

A principled strategy orthogonal to behavior-regularized actor-critic frameworks is a resampling mechanism [17, 20, 32, 33]. Instead of training the action policy with Q value, Max Q sampling leverages the estimated Q_ϕ in the inference time. Formally, given a proposal dis-

tribution $\pi_\theta(a|s)$ and a target criterion $Q_\phi(s, a)$, the Max Q sampling procedure selects the optimal action from a set of samples:

$$a = \operatorname{argmax}_{a^{(1)}, \dots, a^{(n_{\text{sample}})}} Q_\phi(s, a^{(i)}), \quad a^{(i)} \sim \pi_\theta(s) \quad (2.8)$$

2.6 Behavior-Regularized Actor-Critic (BRAC)

Behavior-regularized actor-critic methods form a family of effective offline RL approaches that combine value function learning with behavior regularization [5, 6, 8]. The critic loss follows (D.2). The actor loss, defined in our implementation, integrates policy improvement and behavior regularization:

$$\mathcal{L}_{\pi, \text{BCLoss}}^{\text{BRAC}}(\theta) = \mathbb{E}_{(s,a) \sim \mathcal{D}} \left[-Q_\phi(s, \pi_\theta(s)) + \alpha \cdot \text{BCLoss}(\pi_\theta(s), a) \right], \quad (2.9)$$

where α is a hyperparameter balancing policy improvement and behavior constraint and BCLoss is a behavior clone loss, e.g. $\|\pi_\theta(s) - a\|^2$ or the velocity matching loss in the previous section. While the BARC objective is almost necessary, it has been shown to be highly sensitive to α [25, 34, 35, 51], requiring extensive per-task tuning.

The problem in applying Equation 2.9 to a diffusion/flow-based policy is the computationally costly iterative sampling chain [9–11]. There exists many works to circumvent this issue, see Section 1.4 for more discussion.

Chapter 3 Methodology

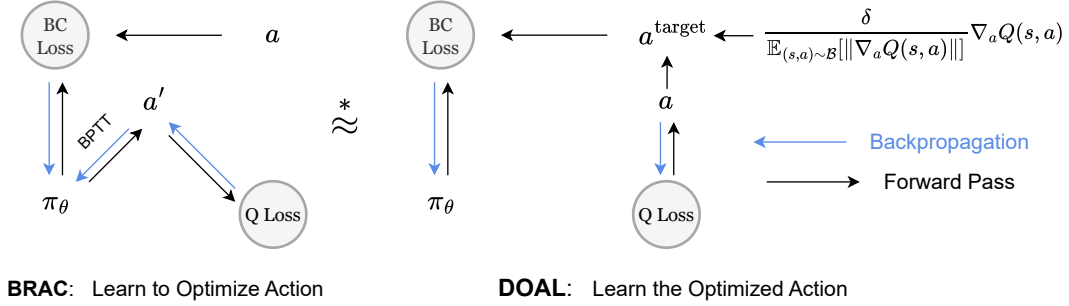


Figure 3.1 DOAL Framework. [LEFT] End2end re-parameterized policy gradient from the Q and the behavior clone loss. [RIGHT] DOAL framework, where we extract an optimized a^{target} from Q, and use policy dependent behavior clone loss.

In deep learning, there is a doctrine that all we need to train neural networks is a differentiable loss w.r.t. the parameters [52]. In the context of learning policy with iterative sampling, the computational cost becomes too high. Moreover, it is not clear what the learned policy represents aside from the fact that it balances behavior regularization and expected return maximization. We present an intuitive motivation and a formal motivation to derive our Direct Optimal Action Learning framework, and show how it can naturally yield more interpretable and robust hyperparameter choice.

3.1 The Optimal Action

3.1.1 Intuitive Perspective.

An actor policy should learn to produce actions that have high estimated Q-values. When learning from a static dataset, this search for high-value actions should naturally be centered around the actions already present in the data. This leads to a straightforward idea of direct optimal action learning (DOAL): instead of indirectly learning a policy that optimizes a complex objective, we can directly learn the optimal action itself. In related works 1.4, we discuss other neural network learning methods that also follow non-conventional target matching objectives.

3.1.2 Formal perspective.

The DOAL idea emerges from a re-interpretation of the policy gradient in standard behavior-regularized actor-critic methods:

Proposition 3.1 (BRAC Objective Pursue BRAC Target). *Let $\pi_\theta(s)$ be a deterministic differentiable policy and $Q_\phi(s, a)$ be a differentiable action-value function. Consider the behavior-regularized objective on data point s, a :*

$$\mathcal{L}_Q(\theta) \triangleq \mathbb{E}_{(s,a) \sim \mathcal{D}} [Q_\phi(s, \pi_\theta(s)) - \alpha \|\pi_\theta(s) - a\|_2^2] \quad (3.1)$$

$$\nabla_\theta \mathcal{L}_{\text{brac_target}}(\theta) \triangleq \mathbb{E}_{(s,a) \sim \mathcal{D}} \left[\nabla_\theta \left(-\alpha \|\pi_\theta(s) - a^{\text{brac_target}}\|_2^2 \right) \right] \quad (3.2)$$

$$a^{\text{brac_target}} = a + \frac{1}{2\alpha} \nabla_{a'} Q_\phi(s, a')|_{a'=\pi_\theta(s)} \quad (3.3)$$

where $\alpha > 0$ is a regularization coefficient. We have:

$$\nabla_\theta \mathcal{L}_Q(\theta) = \nabla_\theta \mathcal{L}_{\text{brac_target}}(\theta) \quad (3.4)$$

The proof is an application of the chain rule (see Appendix A). This gradient equivalence reveals that the BRAC objective implicitly minimizes the distance between the policy's output $\pi_\theta(s)$ and a target action $a^{\text{brac_target}}$. Under such an interpretation, a conceptual inconsistency exists. The target $a^{\text{brac_target}}$ is constructed by taking a gradient ascent step from the data action a , but the gradient $\nabla_{a'} Q_\phi$ is evaluated at the policy's output $\pi_\theta(s)$. This requires action sampling at training time and creates a mismatch between the point of expansion (a) and the point of Q value gradient evaluation ($\pi_\theta(s)$). DOAL performs the gradient ascent using the gradient evaluated at the data action a , which would define a a^{target} for each data point (s, a) without sampling. Formally, we have the DOAL target:

$$a^{\text{DOAL}} = a + \frac{1}{2\alpha} \nabla_a Q_\phi(s, a) \quad (3.5)$$

Remark 3.2. Comparing Equation 3.3 and 3.5, Proposition 3.1 shows BRAC objective and the DOAL objective are *similar but different*. DOAL is a reasonable objective for offline RL in its own right. Interestingly, if we consider the online RL scenario, one could take the policy gradient using the sampled action for actual execution. In such case, we have the equivalence $a = a' \implies a^{\text{DOAL}} = a^{\text{brac_target}}$.

By defining the target action directly from the data and Q value, DOAL decouple the target computation from the policy being trained. The major practical benefit is that we no longer need to sample an action for training the actor.

3.2 Batch-Normalizing Optimizer

A challenge with BRAC-style methods is the sensitivity of the regularization coefficient α , which often requires careful tuning across several orders of magnitude [1, 24]. Our re-interpretation of the objective as learning a target action raises a critical question: what is the appropriate magnitude for the update from the data action a to the target action a^{target} ?

In offline reinforcement learning, the learned Q-function is merely an estimator, and it is crucial to remain conservative to avoid distribution shift to out-of-support actions where the Q-function is unreliable [7, 8, 24]. Therefore, instead of an arbitrary step size dictated by α , we should define a **statistical trust region** for our action optimization. We can achieve this by setting a fixed expected magnitude for the update vector $g(s, a) = a^{\text{target}} - a$. Specifically, we desire two conditions for the update vector $g(s, a)$:

1. The update should be in the direction of the Q-function’s gradient at the data action:

$$g(s, a) = C \cdot \nabla_a Q_\phi(s, a) \text{ where } C > 0.$$

2. The expected squared magnitude of the update over the dataset should be a constant, which we denote as δ : $\mathbb{E}_{(s,a) \sim \mathcal{D}} [\|g(s, a)\|_2] = \delta$.

These two conditions uniquely determine the update vector, as shown in the following proposition.

Proposition 3.3 (Batch Normalized Update). *To satisfy the conditions $g(s, a) = C \cdot \nabla_a Q_\phi(s, a)$ and $\mathbb{E}_{(s,a) \sim \mathcal{D}} [\|g(s, a)\|_2] = \delta$, where $\delta, C > 0$, the only action update vector $g(s, a)$ is :*

$$g(s, a) = \frac{\delta}{E_{(s',a') \sim \mathcal{D}} [\|\nabla_{a'} Q_\phi(s', a')\|_2]} \cdot \nabla_a Q_\phi(s, a) \quad (3.6)$$

The proof is a straightforward calculation (see Appendix B). This formulation replaces the obscure hyperparameter α with an interpretable one, δ , which directly controls the expected squared magnitude of the action update. In practice, we use the batch statistics as estimator, so we have $E_{(s',a') \sim \mathcal{B}} [\|\nabla_{a'} Q_\phi(s', a')\|_2^2]$, where $\mathcal{B}$ is the current mini-batch. This

“batch-normalization” of the action gradients provides a more stable and robust training target, alleviating the need for extensive hyperparameter sweeps. *We are not claiming that this batch-normalized scheme can find better a^{target} than not using batch-normalized gradient.* In fact, if the gradient statistics is stable, you can always get the same result by having $g(s, a) = C \cdot \nabla_a Q_\phi(s, a)$ where $C := \frac{\delta}{E_{(s', a') \sim \mathcal{D}} [\|\nabla_{a'} Q_\phi(s', a')\|_2]}$. We will empirically see that the optimal δ varies less than α .

3.3 The Batch-Normalized DOAL Objectives

DOAL can work with any value function learning. We only alter the actor loss :

$$\mathcal{L}_{\text{DOAL}}(\theta) = \alpha \cdot \mathbb{E}_{(s, a) \sim \mathcal{D}} \text{BCLoss}(\pi_\theta(s), a^{\text{target}}) \quad (3.7)$$

$$a^{\text{target}} := a + \frac{\delta \cdot \nabla_a Q_\phi(s, a)}{E_{(s', a') \sim \mathcal{B}} [\|\nabla_{a'} Q_\phi(s', a')\|_2]} \quad (3.8)$$

where BCLoss can be any distribution matching losses. In terms of computational overhead, as the gradient $\nabla_a Q_\phi(s, a)$ will be called explicit or implicitly at least once to extract first-order structure information from Q_ϕ .

Chapter 4 Experiments

In this section, we examine our DOAL framework. Our baseline models benefit from $Q_\phi(s, a)$ either through advantage weighted regression or max-Q sampling without access to $\nabla Q_\phi(s, a)$. We analyse the time complexity of DOAL, and discuss the trust region δ selection. Other ablation studies are presented in Appendix G.

4.1 Toy Experiment Results

Toy Experiment Setting: To empirically evaluate the efficacy of DOAL against BRAC under extreme data-sparsity conditions, we construct a synthetic benchmark where the ground-truth reward and transition dynamics are governed by a pre-trained, randomized multi-layer perceptron (MLP) Q-network. The continuous action space is strictly bounded within $[-2, 2]$. For the learning agents, we adopt a standard MLP architecture consisting of two hidden layers with 128 hidden units each, interspersed with ReLU activation functions. All policies are optimized for a total of 10,000 training steps.

In Figure 4.1, by observing the row of images for DOAL ($\delta = 0.2$ to $\delta = 2.0$), it is evident that as the radius threshold δ varies, the valid boundaries of the Q -values exhibit a highly distinct local spherical truncation characteristic. Meanwhile, the DOAL preserves the true value landscape (Real Q): regardless of how large δ becomes, within the core regions of the dataset, the Real Q estimated by DOAL consistently maintains a reasonable positive range (e.g., 0.56, 0.47, etc.), closely matching the Ground Truth Q . Consequently, DOAL successfully prevents the value function from expanding uncontrollably in out-of-distribution (OOD) extrapolation regions, while perfectly preserving the true value gradients within the data distribution when the data is sparse.

4.2 Main Experiments Setup

We perform a comprehensive evaluation of the DOAL framework across combinations of value function learning and action policies. We test under implicit Q-learning, Q-learning, and regularized Q-learning. Our policy models include simple Gaussian policy, Multi-step Flow models, and TrigFlow models. Except for the number of MaxQ sampling candidates, we follow the hyperparameters from FQL [1] whenever possible, which includes τ in IQL

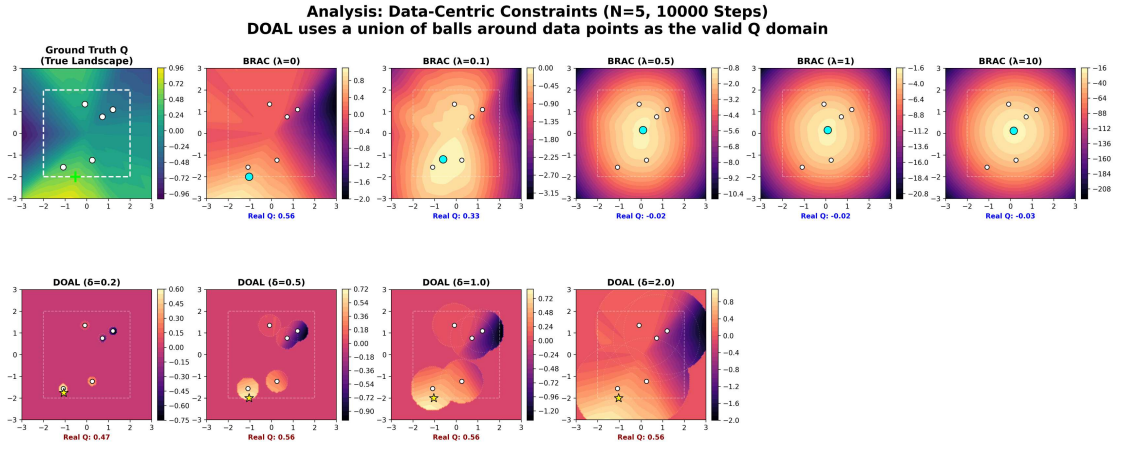


Figure 4.1 Performance ratio under unchanged conditions, where $n=5$

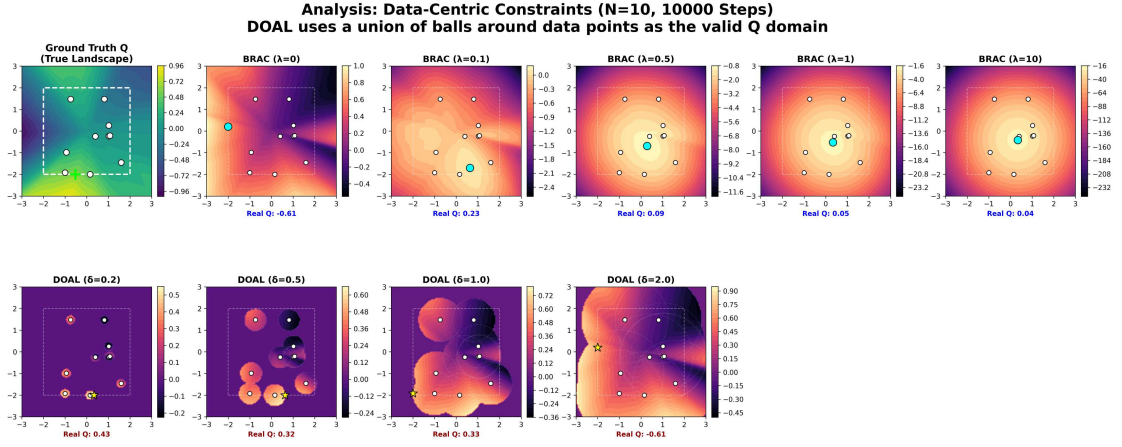


Figure 4.2 Performance ratio under unchanged conditions, where $n=10$

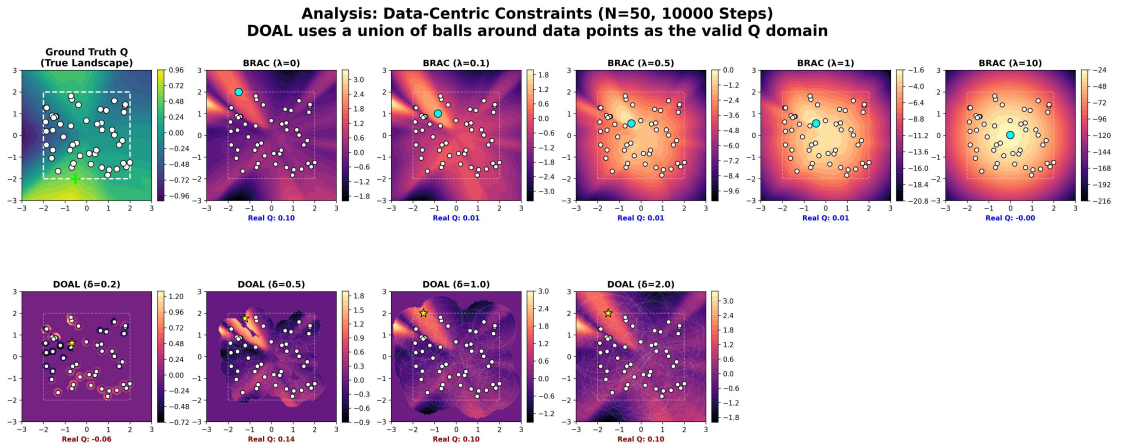
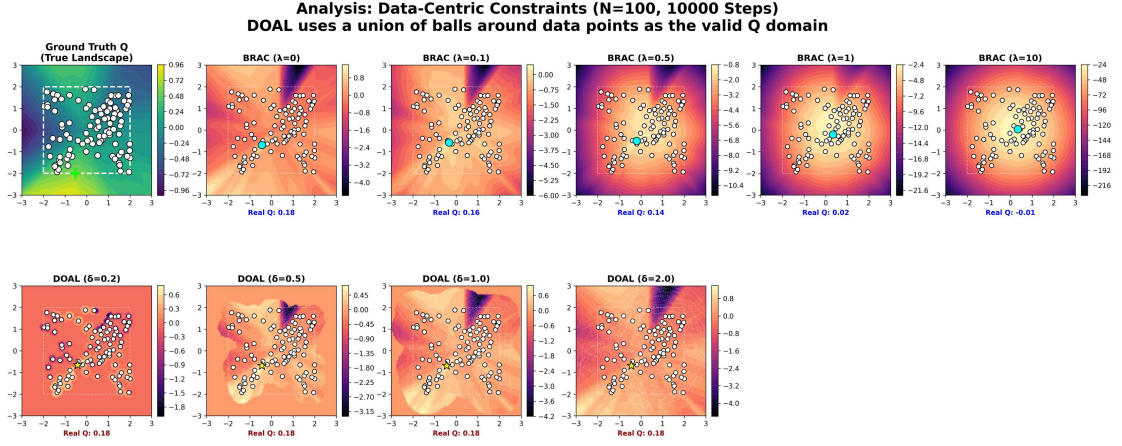
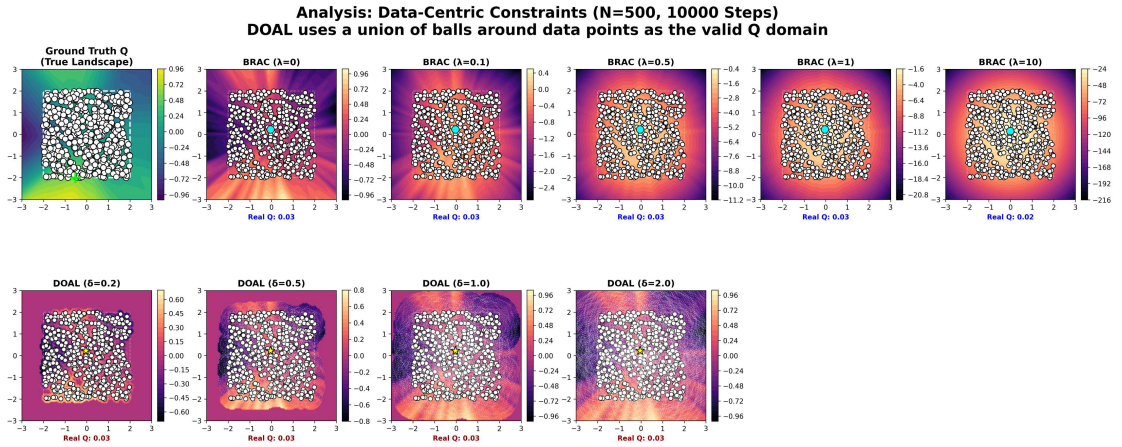


Figure 4.3 Performance ratio under unchanged conditions, where $n=50$

Figure 4.4 Performance ratio under unchanged conditions, where $n=100$ Figure 4.5 Performance ratio under unchanged conditions, where $n=500$

and α for scaling BC loss.

For the IQL value function, we have three representative and strong baseline algorithms with our method: IQL, with a simple Gaussian policy learned from AWR; IFQL, a flow policy with MaxQ sampling; TrigFlow, a trigflow diffusion policy with MaxQ sampling. For each one of them, we have the DOAL versions DIQL, DIFQL, and DTrigflow. Furthermore, we have efficient trigflow (ETrigflow [18, 49], see Appendix E for details) that uses the BRAC objective but uses one-step sampling from corrupted a_t and still uses IQL for value learning.

For the Q-learning value function, our baseline model MFQL replace the IQL value learning with Q-learning. Also, we have the DOAL version DMFQL. Furthermore, we add the ReBRAC objective to regularize the Q learning target, and get MFReBRAC and DMFReBRAC (see Appendix E for details). We also report the results of MFQL_{bptt} that actually running BPTT for actor learning.

4.3 Evaluation and Benchmarks

We conduct a comprehensive evaluation of our method on two offline RL benchmarks and one online benchmark: 1) The 6 Adroit/D4RL tasks with expert, human, and cloned dataset qualities. 2) 9 default tasks from the more diverse and demanding OGBench suite. We omit some tasks, as no current algorithms can work well [1] without considering horizon reduction techniques [33, 53, 54]. Following [1], we train on OGBench for 1m steps, and take the average performance on 800k, 900k, and 1m steps. We train on D4RL tasks for 500k steps and test on the last step. For the preliminary study of DOAL under an online setting, we tested our DMFQL against TD3 [55] on 8 tasks in Brax [56] for 1m step using the Rejax package [57].

4.4 Main Results

Table 4.1 IQL-based offline RL results. Full results on the OGBench and D4RL tasks with IQL value function. They are the default single-task in each environment.

Task	Simple Policies			Flow Policies			Diffusion Policies		
	IQL(tanh)*	IQL(Gauss)	DIQL	IFQL*	IFQL	DIFQL	TrigFlow	DTrigFlow	ETrigFlow
antmaze-large-navigate	-	48 $\pm$ 9	63 $\pm$ 10	24 $\pm$ 17	48 $\pm$ 24	67 $\pm$ 6	72 $\pm$ 6	63 $\pm$ 24	63 $\pm$ 21
humanoidmaze-medium-navigate	-	32 $\pm$ 7	55 $\pm$ 8	69 $\pm$ 19	68 $\pm$ 3	68 $\pm$ 4	64 $\pm$ 4	67 $\pm$ 4	63 $\pm$ 4
humanoidmaze-large-navigate	-	3 $\pm$ 1	10 $\pm$ 3	6 $\pm$ 2	6 $\pm$ 3	8 $\pm$ 3	7 $\pm$ 2	8 $\pm$ 4	6 $\pm$ 3
antsoccer-arena-navigate	-	3 $\pm$ 2	13 $\pm$ 3	16 $\pm$ 9	40 $\pm$ 5	40 $\pm$ 6	40 $\pm$ 8	41 $\pm$ 4	41 $\pm$ 9
cube-single-play	-	85 $\pm$ 8	80 $\pm$ 4	73 $\pm$ 3	88 $\pm$ 4	90 $\pm$ 3	86 $\pm$ 4	88 $\pm$ 2	88 $\pm$ 4
cube-double-play	-	1 $\pm$ 1	3 $\pm$ 2	9 $\pm$ 5	11 $\pm$ 3	21 $\pm$ 4	16 $\pm$ 4	22 $\pm$ 3	16 $\pm$ 3
scene-play	-	12 $\pm$ 3	37 $\pm$ 10	0 $\pm$ 0	40 $\pm$ 23	40 $\pm$ 23	43 $\pm$ 16	46 $\pm$ 15	50 $\pm$ 12
puzzle-3x3-play	-	2 $\pm$ 1	5 $\pm$ 1	0 $\pm$ 0	5 $\pm$ 1	5 $\pm$ 2	7 $\pm$ 2	7 $\pm$ 3	8 $\pm$ 2
puzzle-4x4-play	-	5 $\pm$ 2	10 $\pm$ 2	21 $\pm$ 11	23 $\pm$ 7	21 $\pm$ 5	26 $\pm$ 5	27 $\pm$ 6	26 $\pm$ 4
Total	-	191	276	218	329	359	361	368	359
pen-human-v1	78	54 $\pm$ 6	43 $\pm$ 8	71 $\pm$ 12	81 $\pm$ 8	68 $\pm$ 8	71 $\pm$ 11	69 $\pm$ 13	72 $\pm$ 12
pen-cloned-v1	83	66 $\pm$ 7	56 $\pm$ 9	80 $\pm$ 11	73 $\pm$ 7	74 $\pm$ 7	65 $\pm$ 7	67 $\pm$ 8	67 $\pm$ 9
pen-expert-v1	128	131 $\pm$ 8	132 $\pm$ 4	139 $\pm$ 5	134 $\pm$ 4	138 $\pm$ 4	135 $\pm$ 8	133 $\pm$ 7	134 $\pm$ 8
door-expert-v1	107	104 $\pm$ 2	104 $\pm$ 2	104 $\pm$ 2	104 $\pm$ 1	104 $\pm$ 1	104 $\pm$ 1	104 $\pm$ 1	104 $\pm$ 1
hammer-expert-v1	129	68 $\pm$ 47	76 $\pm$ 46	117 $\pm$ 9	96 $\pm$ 8	98 $\pm$ 12	103 $\pm$ 8	98 $\pm$ 11	100 $\pm$ 10
relocate-expert-v1	106	97 $\pm$ 10	101 $\pm$ 5	104 $\pm$ 3	104 $\pm$ 3	102 $\pm$ 8	106 $\pm$ 2	107 $\pm$ 2	106 $\pm$ 2
Total	631	520	518	615	592	584	584	577	583

Table 4.1 shows we have very strong baseline models of IFQL and Trigflow. The results are averaged over 8 seeds on D4RL and OGBench. The IQL(tanh)*, IQL(Gauss) on OGBench, IFQL* results are collected from Flow Q learning [1, 58]. In particular, our n_{sample} tuned IFQL improved over the original IQL* significantly on OGBench. On the OGBench, on aggregation, our DOAL models performed better than their baselines. Upon closer examination, we find that those are due to one or two tasks that have significant gains that we highlighted. Otherwise, their performance is very similar. On antmaze-large, both DTrigFlow and ETrigFlow dropped their performance. We find that there are two seeds that have very low performance (hence the large std). For training curves, see Appendix F.

On the D4RL, we re-run IQL from the FQL paper. It appears that there is no performance

Table 4.2 Q-Learning based offline RL results. The results are averaged over 8 seeds . The ReBRAC*, FQL* results are collected from Flow Q learning [1].

Task	Simple Policies		Flow Policies					
	ReBRAC(tanh)*	ReBRAC(Gauss)	FQL*	MFQL	DMFQL	MFQL _{bptt}	MFRReBRAC	DMFReBRAC
antmaze-large-navigate	91 $\pm$ 10	-	80 $\pm$ 8	62 $\pm$ 11	72 $\pm$ 8	64 $\pm$ 13	65 $\pm$ 13	83 $\pm$ 7
humanoidmaze-medium-navigate	16 $\pm$ 9	-	19 $\pm$ 12	49 $\pm$ 9	44 $\pm$ 13	54 $\pm$ 9	53 $\pm$ 14	52 $\pm$ 7
humanoidmaze-large-navigate	2 $\pm$ 1	-	7 $\pm$ 6	8 $\pm$ 3	7 $\pm$ 3	6 $\pm$ 2	9 $\pm$ 4	8 $\pm$ 2
antsoccer-arena-navigate	0 $\pm$ 0	-	39 $\pm$ 6	43 $\pm$ 6	37 $\pm$ 5	45 $\pm$ 5	45 $\pm$ 5	41 $\pm$ 6
cube-single-play	92 $\pm$ 4	-	97 $\pm$ 2	95 $\pm$ 1	98 $\pm$ 1	62 $\pm$ 37	91 $\pm$ 5	99 $\pm$ 1
cube-double-play	7 $\pm$ 3	-	36 $\pm$ 6	72 $\pm$ 4	75 $\pm$ 6	72 $\pm$ 3	74 $\pm$ 4	75 $\pm$ 3
scene-play	50 $\pm$ 13	-	76 $\pm$ 9	57 $\pm$ 20	90 $\pm$ 10	68 $\pm$ 15	57 $\pm$ 12	92 $\pm$ 6
puzzle-3x3-play	2 $\pm$ 1	-	16 $\pm$ 5	7 $\pm$ 3	6 $\pm$ 2	1 $\pm$ 1	7 $\pm$ 2	5 $\pm$ 2
puzzle-4x4-play	10 $\pm$ 3	-	11 $\pm$ 3	24 $\pm$ 3	14 $\pm$ 4	0 $\pm$ 0	25 $\pm$ 5	12 $\pm$ 3
Total	297	-	381	418	443	372	425	466
pen-human-v1	103	55 $\pm$ 9	53 $\pm$ 6	75 $\pm$ 9	72 $\pm$ 8	-	64 $\pm$ 9	74 $\pm$ 8
pen-cloned-v1	92	72 $\pm$ 15	74 $\pm$ 11	75 $\pm$ 9	80 $\pm$ 5	-	71 $\pm$ 12	75 $\pm$ 10
pen-expert-v1	152	143 $\pm$ 6	142 $\pm$ 6	138 $\pm$ 4	130 $\pm$ 8	-	140 $\pm$ 9	143 $\pm$ 4
door-expert-v1	106	105 $\pm$ 1	104 $\pm$ 1	104 $\pm$ 2	104 $\pm$ 1	-	105 $\pm$ 8	105 $\pm$ 1
hammer-expert-v1	134	131 $\pm$ 1	125 $\pm$ 3	126 $\pm$ 3	124 $\pm$ 5	-	126 $\pm$ 3	126 $\pm$ 1
relocate-expert-v1	108	107 $\pm$ 1	107 $\pm$ 1	106 $\pm$ 4	104 $\pm$ 4	-	106 $\pm$ 1	107 $\pm$ 2
Total	706	614	605	623	614	-	614	630

gain from either the DOAL model or even ETrigflow. This might be due to the unreliability of the IQL learned function gradient. It should be noted that the DOAL models subsume their baselines, as one can set $\delta = 0$ to **recover** the baseline or choose an extremely small δ . However, even if finer search for δ can yield higher performance, one can not rule out the selection bias. A proper way to address the inability to extract information from Q value is to investigate (Regularized) Q-Learning-based functions. As for α weights on actor loss, we still keep the α parameter from [1] for all experiments for consistency. In Appendix G, we set $\alpha = 1$ and find no difference.

In Table 4.2, excluding MFQL with BPTT, all our models outperform FQL. In fact, in Table 4.2, MFQL outperforms FQL. This shows the effectiveness of MaxQ sampling. Furthermore, DMFQL outperforms MFQL on OGBench but not on D4RL. This again indicates that the effectiveness of DOAL might depend on the task or the quality of Q value function. The fact that DMFReBRAC outperforms MFRReBRAC indicates well regularized Q function can make DOAL work better. We present MFQL with BPTT to show that while being less stable, it is not always weaker. However, the computational complexity of BPTT is indeed much higher, as we will discuss.

In Figure 4.8, we have the relations between MFQL, DMFQL, MFRReBRAC, and DMFReBRAC. We should notice that the DOAL version can achieve the same result as their baselines by letting $\delta = 0$, but we do not include such a choice to explicitly show that first-order gradient-based policy extraction might not always work.

4.5 Discussion

4.5.1 Importance of Regularization

On D4RL tasks in Table 4.2, we observe that only the regularized Q function can boost the DOAL model performance. This strongly suggests that DOAL or maybe other gradient based policy extraction methods need the Q function to be reliable.

4.5.2 Importance of Tanh

Our models are still lagging behind ReBRAC, that use a simple policy. We identified another difference, which is that the ReBRAC used tanh nonlinearity for producing actions between $[-1,1]$. This might introduce inductive bias. Indeed, by removing it, the performance of ReBRAC dropped. Studying how to add this nonlinear transformation after flow models is an interesting research question.

4.5.3 Time Complexity

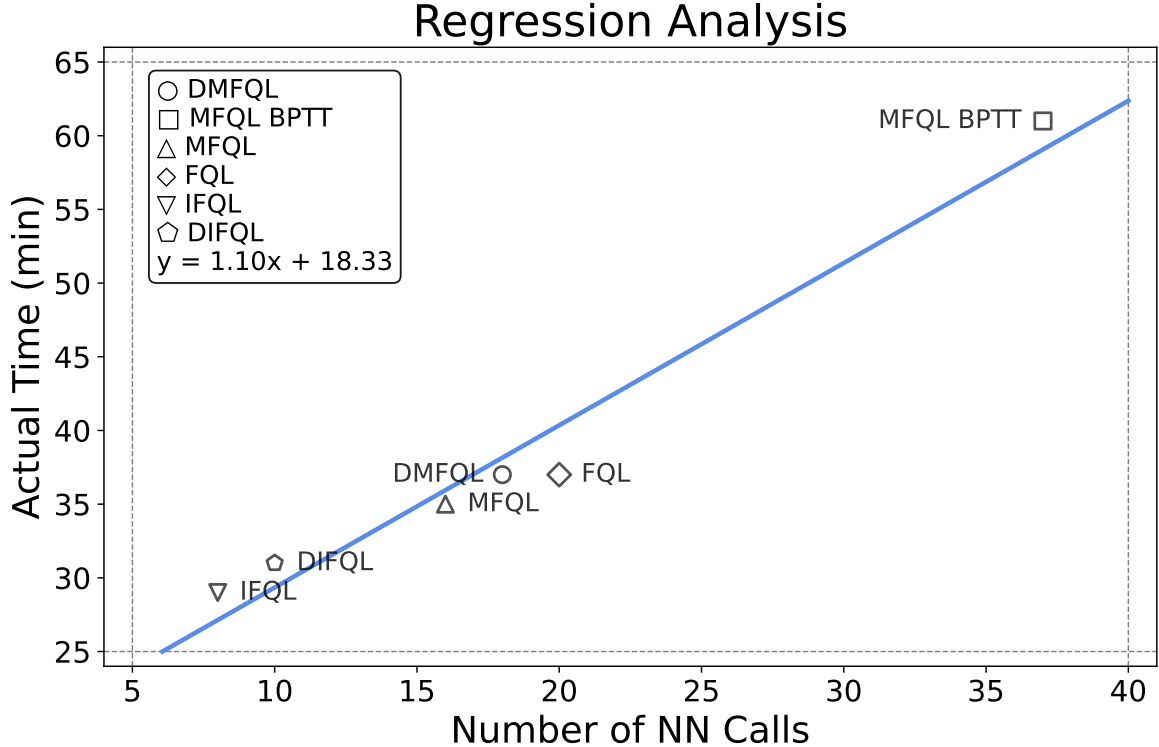


Figure 4.6 Runtime and Computational Complexity . A regression line models the actual running time by the number of function calls. The actual run time is on the antmaze-large with an A800.

In this section, 4-layer MLPs are used for all neural modules. In Figure 4.6, we count the

number of forward and backward calls through value and policy networks and present their total calls during the training phase. As forward and backward calls have similar computation cost and policy and value network share similar architectures, the sum of total calls can be used to predict the actual run time. Indeed, in Figure 4.6, an affine relation is found. The relation is not linear, because there are overheads such as data loading and testing. One might notice that FQL’s actual run time is faster than predicted; this is because FQL uses a one-step policy during testing. We did not include DMFReBRAC as it is the same as DMFQL. As for the memory usage, it is bottlenecked by the backward policy calls that require storing intermediate computing states. Therefore, only the BPTT version consumes more memory.

4.5.4 Choose Statistical Trust Region δ

In OGBench [59], actions are bounded in a $[-1, 1]$ box of varying dimensionality. The statistical trust region should be related to how reliable the Q value estimation is at a data point and how well the neural network generalizes. As we are using IQL for value estimation and the same value net in all our experiments, the only thing varying is the datasets. For OGBench experiments, we choose δ from $(0.03, 0.1, 0.3)$, and for D4RL experiments, we choose from $(0.0003, 0.001, 0.003)$ as we see from α in the FQL paper [1] tends to be extremely large. See Appendix H for the exact hyperparameters.

Table 4.3 Different Environments with the Observed Mean $\|\nabla_a Q(s, a)\|$, α , and δ .

Envs	puzzle-4x4	cube-single	scene-play	antmaze-large-navigate
$\mathbb{E}_{(s', a') \sim \mathcal{B}} [\ \nabla_{a'} Q_\phi(s', a')\ _2]$	43.78	5.85	15.58	0.55
α	1000	300	300	10
δ	0.03	0.03	0.1	0.1
$\frac{\delta}{\mathbb{E}[\ \nabla Q\]}$	6.9e-4	5.1e-3	1.9e-3	1.8e-2

In Table 4.3, we present a few representative environments and their average $\|\nabla_a Q(s, a)\|$, optimal α [1] and δ we selected. As you can see, the larger the gradient, the larger the selected α . Based on our Proposition 3.1, we would be expecting α be inverse proportional to $\frac{\delta}{\mathbb{E}_{(s', a') \sim \mathcal{B}} [\|\nabla_{a'} Q_\phi(s', a')\|_2]}$. Indeed, while α ranges across two orders of magnitude, our hyperparameter δ is relatively stable and easier to search for.

In Figure 4.7, we can see that the batch gradient norm is quite stable during training. This implies that $\frac{\delta}{\mathbb{E}_{(s', a') \sim \mathcal{B}} [\|\nabla_{a'} Q_\phi(s', a')\|_2]}$ is roughly a constant for each task, therefore, one can equivalently treating the direct gradient scaling factor as a hyperparameter and avoid

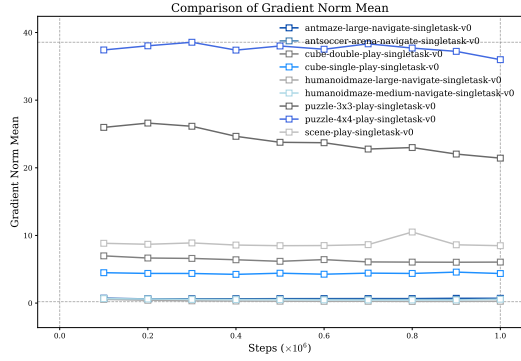


Figure 4.7 Mean Batch Normalized Gradient Norm of DMFQL across OGBench

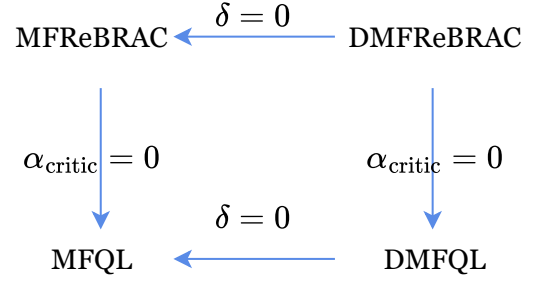


Figure 4.8 The relationship between MFQL, DMFQL, MFReBRAC and DMFReBRAC.

the batch-normalization. The performance would be equivalent. However, the range of this linear scaling hyperparameter would be much wider. In Table 4.3, it ranges across two orders of magnitude just like α .

Chapter 5 Conclusion and Future Work

5.1 Conclusion

In this dissertation, we have systematically investigated the critical challenges of value-based offline reinforcement learning (RL) and presented a novel, unified solution named **Direct Optimal Action Learning**. Offline reinforcement learning serves as a pivotal paradigm for scaling data-driven decision-making to real-world applications, as it circumvents the safety hazards, financial expenses, and logistical constraints associated with active online exploration. However, the absence of continuous environment interaction introduces the fundamental vulnerability of distributional shift. When conventional policy extraction algorithms or value-estimation techniques navigate outside the support of the static behavioral dataset, function approximation errors inevitably compound, causing catastrophic value overestimation and subsequent policy collapse.

To resolve these long-standing vulnerabilities, we present Direct Optimal Action Learning, a framework that enables efficient and effective learning from $\nabla_a Q(s, a)$ for any policy distribution with different Q value functions. We provide strong baselines by re-examining the importance of n_{sample} in MaxQ sampling, then we are able to show our models improved over baseline for OGBench tasks for various policies and value functions. On Adroit tasks, improvements are observed when we use ReBRAC objective. In conclusion, this dissertation provides both a robust theoretical foundation and a highly reproducible engineering implementation, pushing the boundary of data-driven autonomous agents toward reliable real-world deployment.

5.2 Future Work

Although the DOAL framework has established superior performance bounds and robust convergence properties within static offline settings, it opens up several profound and highly promising avenues for future academic and practical research. We structure these prospective research trajectories into two core methodology dimensions.

5.2.1 Algorithmic Extension to Online Adaptation

The current mathematical and empirical foundations of DOAL are strictly established under the offline setting, where the behavioral dataset $\mathcal{D}$ is fixed and invariant. However, the ultimate utility of a robust offline pre-trained policy lies in its capability to serve as a high-quality initialization for subsequent online adaptation.

Preliminary Online Results To evaluate the online performance of our DOAL, we conducted a comparative analysis between DMFQL and the TD3 baseline across eight diverse environments within the Brax suite. For the controlled study, we only changed the policy loss. For these experiments, we set the weighting of the diffusion loss $\alpha = 1$, $\delta = 0.2$, and all hyperparameters were kept consistent with the Rejax implementation of TD3 across all environments. In Figure 5.1, our preliminary results demonstrate that DMFQL can be effectively deployed in online settings without extensive per-environment tuning.

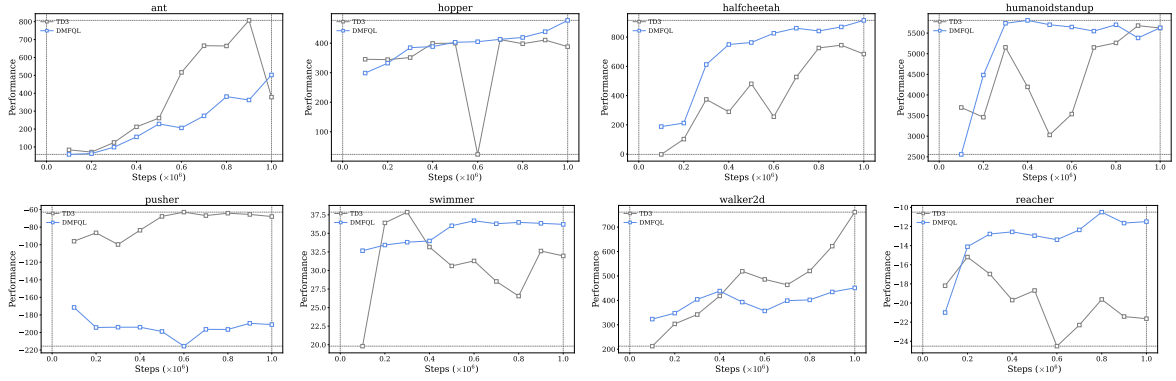


Figure 5.1 Performance of DMFQL and TD3 on the Brax Environments in 1 seed.

5.2.2 Uncertainty-Aware N -Ensemble Mechanics

From a hyperparameter engineering perspective, the optimization landscape of DOAL relies heavily on a precise calibration of the conservative regularization coefficient, δ . This scalar parameter dictates the global strictness of the penalty applied to OOD actions. In standard offline RL workflows, selecting an optimal δ requires exhaustive grid searches.

To completely eliminate this manual bottleneck and elevate the framework’s autonomy, a critical future directive is to substitute the rigid scalar δ with a dynamic, localized uncertainty metric explicitly inspired by the **Diversified Q-Ensemble (EDAC)** framework [25]. Standard ensemble mechanisms typically rely on random initialization and bootstrapping to

generate model disagreement. However, in deep offline RL, neural networks frequently co-adapt and suffer from variance collapse during prolonged Bellman updates, failing to maintain high-variance penalties on critical OOD actions unless an impractically large ensemble size (e.g., $N = 500$) is deployed.

To overcome this, we propose integrating the core design philosophy of EDAC into DOAL by maintaining an ensemble of N distinct Q-networks $\{Q_{\theta_1}, \dots, Q_{\theta_N}\}$ that are penalized via a **gradient diversification objective**.

Consequently, the localized epistemic uncertainty $U(s, a)$, measured by the empirical variance of the diversified ensemble, serves as a highly sensitive, state-action-dependent penalty:

$$U(s, a) = \frac{1}{N-1} \sum_{i=1}^N (Q_{\theta_i}(s, a) - \bar{Q}(s, a))^2, \quad \text{where } \bar{Q}(s, a) = \frac{1}{N} \sum_{i=1}^N Q_{\theta_i}(s, a) \quad (5.1)$$

By incorporating $U(s, a)$ directly into the DOAL objective function, the artificial hyperparameter δ is completely eliminated. In familiar states, $U(s, a) \rightarrow 0$, allowing DOAL to unrestrictedly maximize policy performance; in highly speculative OOD areas, the EDAC-driven gradient diversification causes the value disagreement to explode, automatically enforcing a severe, self-adaptive conservative constraint. Formally integrating this diversified ensemble mechanics will transition DOAL into a sample-efficient, computationally scalable, and entirely parameter-free offline RL paradigm.

References

- [1] S. Park, Q. Li, and S. Levine, “Flow q-learning,” in *Forty-second International Conference on Machine Learning (ICML)*, 2025.
- [2] S. Lange, T. Gabel, and M. Riedmiller, *Batch Reinforcement Learning*. Springer Berlin Heidelberg, 2012, pp. 45–73.
- [3] S. Levine, A. Kumar, G. Tucker, and J. Fu, “Offline reinforcement learning: Tutorial, review, and perspectives on open problems,” *ArXiv*, vol. abs/2005.01643, 2020.
- [4] S. Park, K. Frans, S. Levine, and A. Kumar, “Is value learning really the main bottleneck in offline RL?” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2024.
- [5] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *International conference on machine learning (ICML)*, 2018.
- [6] Y. Wu, G. Tucker, and O. Nachum, “Behavior regularized offline reinforcement learning,” *ArXiv*, vol. abs/1911.11361, 2019.
- [7] I. Kostrikov, A. Nair, and S. Levine, “Offline reinforcement learning with implicit q-learning,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [8] D. Tarasov, V. Kurenkov, A. Nikulin, and S. Kolesnikov, “Revisiting the minimalist approach to offline reinforcement learning,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [9] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics,” in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015.
- [10] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” *Advances in neural information processing systems (NeurIPS)*, 2020.

- [11] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-based generative modeling through stochastic differential equations,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [12] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- [13] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, “Dpm-solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps,” *Advances in neural information processing systems (NeurIPS)*, 2022.
- [14] Q. Sun, Z. Jiang, H. Zhao, and K. He, “Is noise conditioning necessary for denoising generative models?” in *Forty-second International Conference on Machine Learning (ICML)*, 2025.
- [15] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, “Planning with diffusion for flexible behavior synthesis,” in *International Conference on Machine Learning (ICML)*, 2022.
- [16] Z. Wang, J. J. Hunt, and M. Zhou, “Diffusion policies as an expressive policy class for offline reinforcement learning,” in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- [17] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine, “Idql: Implicit q-learning as an actor-critic method with diffusion policies,” *ArXiv*, vol. abs/2304.10573, 2023.
- [18] B. Kang, X. Ma, C. Du, T. Pang, and S. Yan, “Efficient diffusion policies for offline reinforcement learning,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [19] S. Fujimoto and S. Gu, “A minimalist approach to offline reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [20] H. Chen, C. Lu, C. Ying, H. Su, and J. Zhu, “Offline reinforcement learning via high-fidelity generative behavior modeling,” in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

- [21] C. Lu, H. Chen, J. Chen, H. Su, C. Li, and J. Zhu, “Contrastive energy prediction for exact energy-guided diffusion sampling in offline reinforcement learning,” in *International Conference on Machine Learning (ICML)*, 2023.
- [22] D. Garg, J. Hejna, M. Geist, and S. Ermon, “Extreme q-learning: Maxent RL without entropy,” in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- [23] H. Xu, L. Jiang, J. Li, Z. Yang, Z. Wang, V. W. K. Chan, and X. Zhan, “Offline RL with no OOD actions: In-sample learning via implicit value regularization,” in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- [24] A. Kumar, A. Zhou, G. Tucker, and S. Levine, “Conservative q-learning for offline reinforcement learning,” *Advances in neural information processing systems (NeurIPS)*, 2020.
- [25] G. An, S. Moon, J.-H. Kim, and H. O. Song, “Uncertainty-based offline reinforcement learning with diversified q-ensemble,” in *Advances in Neural Information Processing Systems (NeurIPS)*, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., 2021.
- [26] A. Nair, A. Gupta, M. Dalal, and S. Levine, “Awac: Accelerating online reinforcement learning with offline datasets,” 2021.
- [27] S. Fujimoto, D. Meger, and D. Precup, “Off-policy deep reinforcement learning without exploration,” in *International Conference on Machine Learning (ICML)*, 2018.
- [28] H. Chen, K. Zheng, H. Su, and J. Zhu, “Aligning diffusion behaviors with q-functions for efficient continuous control,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [29] S. Ding, K. Hu, Z. Zhang, K. Ren, W. Zhang, J. Yu, J. Wang, and Y. Shi, “Diffusion-based reinforcement learning via q-weighted variational policy optimization,” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

- [30] K. Frans, S. Park, P. Abbeel, and S. Levine, “Diffusion guidance is a controllable policy improvement operator,” 2025.
- [31] X. B. Peng, A. Kumar, G. Zhang, and S. Levine, “Advantage-weighted regression: Simple and scalable off-policy reinforcement learning,” *ArXiv*, vol. abs/1910.00177, 2019.
- [32] S. K. S. Ghasemipour, D. Schuurmans, and S. S. Gu, “Emaq: Expected-max q-learning operator for simple yet effective offline and online rl,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.
- [33] Q. Li, Z. Zhou, and S. Levine, “Reinforcement learning with action chunking,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. [Online]. Available: <https://openreview.net/forum?id=XUks1Y96NR>
- [34] L. Fang, R. Liu, J. Zhang, W. Wang, and B. Jing, “Diffusion actor-critic: Formulating constrained policy iteration as diffusion noise regression for offline reinforcement learning,” in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [35] C.-X. Gao, C. Wu, M. Cao, C. Xiao, Y. Yu, and Z. Zhang, “Behavior-regularized diffusion policy optimization for offline reinforcement learning,” in *Forty-second International Conference on Machine Learning (ICML)*, 2025.
- [36] Anonymous, “Flow actor-critic for offline reinforcement learning,” in *Submitted to The Fourteenth International Conference on Learning Representations*, 2025, under review. [Online]. Available: <https://openreview.net/forum?id=wuncwN7iZN>
- [37] S. Park, D. Ghosh, B. Eysenbach, and S. Levine, “HIQL: Offline goal-conditioned RL with latent states as actions,” in *Thirty-seventh Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [38] S. Park, K. Frans, D. Mann, B. Eysenbach, A. Kumar, and S. Levine, “Horizon reduction makes rl scalable,” 2025.
- [39] H. Hoppe, L. Baty, L. Bouvier, A. Parmentier, and M. Schiffer, “Structured reinforcement learning for combinatorial decision-making,” in *The Thirty-ninth*

- Annual Conference on Neural Information Processing Systems*, 2025. [Online]. Available: <https://openreview.net/forum?id=GS9o7u5njS>
- [40] M. Blondel, A. F. Martins, and V. Niculae, “Learning with fenchel-young losses,” *Journal of Machine Learning Research*, vol. 21, no. 35, pp. 1–69, 2020. [Online]. Available: <http://jmlr.org/papers/v21/19-021.html>
- [41] D.-H. Lee, S. Zhang, A. Fischer, and Y. Bengio, “Difference target propagation,” in *ECML/PKDD*, 2014. [Online]. Available: <https://api.semanticscholar.org/CorpusID:14218948>
- [42] A. Meulemans, F. S. Carzaniga, J. A. K. Suykens, J. Sacramento, and B. F. Grewe, “A theoretical framework for target propagation,” *ArXiv*, vol. abs/2006.14331, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:220055614>
- [43] A. Behrouz, M. Razaviyayn, P. Zhong, and V. Mirrokni, “Nested learning: The illusion of deep learning architectures,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. [Online]. Available: <https://openreview.net/forum?id=nbMeRvNb7A>
- [44] Z. Wang, L. He, C. Lyu, and S. B. Cohen, “Nonparametric learning of two-layer reLU residual units,” *Transactions on Machine Learning Research*, 2022. [Online]. Available: <https://openreview.net/forum?id=YiOI0vqJ0n>
- [45] R. Sutton and A. Barto, “Reinforcement learning: An introduction,” *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 1054–1054, 1998.
- [46] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, “Playing atari with deep reinforcement learning,” *CoRR*, vol. abs/1312.5602, 2013. [Online]. Available: <http://arxiv.org/abs/1312.5602>
- [47] N. Marshall, K. L. Xiao, A. Agarwala, and E. Paquette, “To clip or not to clip: the dynamics of SGD with gradient clipping in high-dimensions,” in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

- [48] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu, and I. Gat, “Flow matching guide and code,” *arXiv preprint arXiv:2412.06264*, 2024.
- [49] C. Lu and Y. Song, “Simplifying, stabilizing and scaling continuous-time consistency models,” in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [50] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [51] T. Chen, Z. Wang, and M. Zhou, “Diffusion policies creating a trust region for offline reinforcement learning,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [52] D. E. Rumelhart, R. Durbin, R. Golden, and Y. Chauvin, *Backpropagation: the basic theory*. USA: L. Erlbaum Associates Inc., 1995, p. 1–34.
- [53] R. Wang, S. S. Du, L. F. Yang, and S. M. Kakakde, “Is long horizon rl more difficult than short horizon rl?” in *Proceedings of the 34th International Conference on Neural Information Processing Systems (NeurIPS)*, 2020.
- [54] Q. Li, S. Park, and S. Levine, “Decoupled q-chunking,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.10926>
- [55] S. Fujimoto, H. Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *International Conference on Machine Learning*, 2018, pp. 1582–1591.
- [56] C. D. Freeman, E. Frey, A. Raichuk, S. Girgin, I. Mordatch, and O. Bachem, “Brax - a differentiable physics engine for large scale rigid body simulation,” 2021. [Online]. Available: <http://github.com/google/brax>
- [57] J. Liesen, C. Lu, and R. Lange, “rejax,” 2024. [Online]. Available: <https://github.com/kerajli/rejax>
- [58] D. Tarasov, A. Nikulin, D. Akimov, V. Kurenkov, and S. Kolesnikov, “CORL: Research-oriented deep offline reinforcement learning library,” in *3rd Offline RL*

- Workshop: Offline RL as a "Launchpad"*, 2022. [Online]. Available: <https://openreview.net/forum?id=SyAS49bBcv>
- [59] S. Park, K. Frans, B. Eysenbach, and S. Levine, "OGBench: Benchmarking offline goal-conditioned RL," in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [60] D. Brandfonbrener, W. F. Whitney, R. Ranganath, and J. Bruna, "Offline RL without off-policy evaluation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [61] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.
- [62] D. Hendrycks and K. Gimpel, "Gaussian error linear units (gelus)," 2023. [Online]. Available: <https://arxiv.org/abs/1606.08415>

Appendix A Proof of Proposition 3.1

Proof. The proof follows from applying the chain rule. First, we compute the gradient of the regularized Q-objective in (3.1):

$$\begin{aligned}\nabla_{\theta} J_Q(\theta) &= \nabla_{\theta} Q(s, \pi_{\theta}(s)) - \nabla_{\theta} (\alpha \|\pi_{\theta}(s) - a\|_2^2) \\ &= \left(\nabla_a Q(s, a') \Big|_{a'=\pi_{\theta}(s)} \right)^{\top} \nabla_{\theta} \pi_{\theta}(s) - 2\alpha (\pi_{\theta}(s) - a)^{\top} \nabla_{\theta} \pi_{\theta}(s) \\ &= \left[\nabla_a Q(s, a') \Big|_{a'=\pi_{\theta}(s)} - 2\alpha (\pi_{\theta}(s) - a) \right]^{\top} \nabla_{\theta} \pi_{\theta}(s)\end{aligned}$$

Next, let the target action be $a^* \triangleq a + \frac{1}{2\alpha} \nabla_a Q(s, a') \Big|_{a'=\pi_{\theta}(s)}$. We compute the gradient of the target-matching objective $J_{\text{target}}(\theta) = -\alpha \|\pi_{\theta}(s) - a^*\|_2^2$:

$$\begin{aligned}\nabla_{\theta} J_{\text{target}}(\theta) &= -\nabla_{\theta} (\alpha \|\pi_{\theta}(s) - a^*\|_2^2) \\ &= -2\alpha (\pi_{\theta}(s) - a^*)^{\top} \nabla_{\theta} \pi_{\theta}(s) \\ &= -2\alpha \left(\pi_{\theta}(s) - \left(a + \frac{1}{2\alpha} \nabla_a Q(s, a') \Big|_{a'=\pi_{\theta}(s)} \right) \right)^{\top} \nabla_{\theta} \pi_{\theta}(s) \\ &= \left[-2\alpha (\pi_{\theta}(s) - a) + \nabla_a Q(s, a') \Big|_{a'=\pi_{\theta}(s)} \right]^{\top} \nabla_{\theta} \pi_{\theta}(s)\end{aligned}$$

The resulting gradients are identical. □

Appendix B Proof of Proposition 3.3

Proof. We are given two conditions that $g(s, a)$ must satisfy:

1. $g(s, a) \propto \nabla_a Q_\phi(s, a)$
2. $\mathbb{E}_{(s,a) \sim \mathcal{D}}[\|g(s, a)\|_2] = \delta$

From the first condition (proportionality), we can write $g(s, a)$ as the gradient scaled by some constant C :

$$g(s, a) = C \cdot \nabla_a Q_\phi(s, a)$$

The constant C must be determined. We use the second condition to solve for C .

We substitute our expression for $g(s, a)$ into the second condition:

$$\begin{aligned}\mathbb{E}_{(s,a) \sim \mathcal{D}}[\|g(s, a)\|_2] &= \delta \\ \mathbb{E}_{(s,a) \sim \mathcal{D}}[\|C \cdot \nabla_a Q_\phi(s, a)\|_2] &= \delta\end{aligned}$$

Using the properties of norms, we can pull the scalar C out of the L2-norm as its absolute value:

$$\mathbb{E}_{(s,a) \sim \mathcal{D}}[C \cdot \|\nabla_a Q_\phi(s, a)\|_2] = \delta$$

Because C is a constant, we can pull it out of the expectation:

$$C \cdot \mathbb{E}_{(s,a) \sim \mathcal{D}}[\|\nabla_a Q_\phi(s, a)\|_2] = \delta$$

Now, we solve for C :

$$C = \frac{\delta}{\mathbb{E}_{(s,a) \sim \mathcal{D}}[\|\nabla_a Q_\phi(s, a)\|_2]}$$

Finally, we substitute this expression for C back into our original equation for $g(s, a)$. To avoid ambiguity with the variables of integration in the expectation, we use (s', a') as dummy

variables for the expectation, as shown in the proposition:

$$g(s, a) = \left(\frac{\delta}{\mathbb{E}_{(s', a') \sim \mathcal{D}} [\|\nabla_{a'} Q_\phi(s', a')\|_2]} \right) \cdot \nabla_a Q_\phi(s, a)$$

This completes the proof. □

Appendix C MaxQ Sampling needs Balancing

As our DOAL framework learn from optimized action based on $\nabla_a Q(s, a)$, we consider models without learning from $\nabla_a Q(s, a)$ as our baseline models. In such case, $Q(s, a)$ can still be used for weighting the behavior clone loss and performing actions selection, as we discussed Section 2.5. To have a solid baseline for comparison, we argue that n_{sample} a crucial hyper-parameter. While [33, 54] has pointed out the importance of this hyperparameter, we provide an additional argument here to justify the necessity of tuning n_{sample} .

When taking $n_{\text{sample}} = 1$, we clearly recover the actor distribution and completely lost access to information from Q_ϕ . On the other extreme, Some earlier work suggested the bigger n_{sample} , the better [32]. However, while a^* maximize the Q value estimator Q_ϕ as $n_{\text{sample}} \rightarrow +\infty$, there exists maximization bias such that the maximum Q value will be overestimated due to noise in the estimator, and a^* with large positive random fluctuation will be selected.

Proposition C.1 (Informal). *Consider countably many actions $a_1, a_2, \dots$. For each i , the Q-estimator is independent Gaussian: $\hat{Q}(a_i) \sim \mathcal{N}(\mu_i, \sigma_i^2)$, with bounded means $\sup_i \mu_i \leq U < \infty$ and nontrivial noise $\sigma_i \geq c > 0$. Draw n i.i.d. actions from any policy with dense support and pick the one with the largest observed $\hat{Q}$. As $n \rightarrow \infty$:*

- (i) the selected $\hat{Q}$ value diverges to $+\infty$ (driven by extreme positive noise);
- (ii) the probability of picking any true-mean maximizer tends to 0.

Proof Intuition. The maximum of m i.i.d. Gaussian draws grows like $\mu_i + \sigma_i \sqrt{2 \log m}$. With infinitely many actions having $\sigma_i > 0$, actions with an extremely large positive fluctuation will dominate the max, regardless of the bounded means. Thus, making n_{sample} large pushes selection toward noise outliers rather than actions with the highest μ_i , even with unbiased estimator. $\square$

The above analysis shows that increasing n_{sample} can exacerbate the maximization bias: as $n_{\text{sample}} \rightarrow \infty$, max-selection over noisy Q-estimates systematically prefers actions with large positive noise realizations, independent of the learned Q . While it is hard to precisely characterize the resulting distribution from MaxQ sampling, the more samples we have the more data coverage we have. In the case where action policy is value agnostic, having multiple

samples is necessary to ensure mode coverage so that Q function can provide a selection. Yet, if we sample too much, the stochasticity in Q_ϕ dominates and we might get “good” actions due to overestimation rather than being reliable. Pragmatically, n_{sample} balances the conflict between the in-distribution behavior and the noisy Q value estimator in the inference time.

Appendix D Value Functions

Implicit Q-Learning. IQL [7] avoids querying out-of-sample actions through expectile regression. Unlike SARSA-style methods [60] that learn using the next actions from the dataset, IQL learns the value function of the current policy.

The value function $V_\psi(s)$ and the Q-function Q_ϕ are learned via:

$$\mathcal{L}_V(\psi) = \mathbb{E}_{(s,a) \sim \mathcal{D}} [L_2^\tau(Q_\phi(s, a) - V_\psi(s))], \quad (\text{D.1})$$

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(r + \gamma V_\psi(s') - Q_\phi(s, a))^2 \right] \quad (\text{D.2})$$

where $L_2^\tau(u) = |\tau - \mathbb{I}(u < 0)|u^2$. For policy extraction, standard IQL uses Advantage-Weighted Regression (AWR) [31]:

$$\mathcal{L}_{\text{AWR}}(\theta) = \mathbb{E}_{(s,a) \sim \mathcal{D}} [\exp(\beta(Q_\phi(s, a) - V_\psi(s))) \log \pi_\theta(a|s)], \quad (\text{D.3})$$

where β controls the strength of the advantage weighting.

Appendix E DOAL Objectives

In summary, the overall algorithm for DOAL is given in Algorithm 1. Similarly, you can replace the line 2 in Algorithm 1 with any Q value learning algorithms. In this section, we explicitly discuss the DOAL objectives. If one replace a^{target} with dataset action a , one recover the baseline model.

Algorithm 1 Direct Optimal Action Learning (DOAL)

- 1: **Input:** Dataset $\mathcal{D}$, policy parameters θ , Q-function parameters ϕ [, Value-function parameters ψ]
 - 2: **repeat**
 - 3: Update ϕ, ψ using value loss of Choice
 - 4: Update θ using Equation 3.7 for BCLoss of Choice
 - 5: **until** convergence
-

E.1 Directed Implicit Q-Learning (DIQL)

DIQL extends the implicit Q-learning framework by introducing a direct optimal action learning approach that explicitly guides policy optimization through value-aware action adjustments. The key innovation lies in replacing the action of the data set a with the optimized target action a^* in the policy loss, thus directly steering the policy towards high-value regions.

$$\mathcal{L}_{\pi}^{\text{DIQL}}(\theta) = \mathbb{E}_{(s,a) \sim \mathcal{D}} \left[\exp(\alpha(Q_{\phi}(s, a) - V_{\psi}(s))) \log \pi_{\theta}(a^{\text{target}}|s) \right], \quad (\text{E.1})$$

Notice that the weighting uses the original $Q(s, a)$ as we want to reduce computational overhead.

E.2 Direct Implicit Flow-Q-Learning (DIFQL)

Implicit Flow-Q-Learning (DIFQL) [1] builds upon the framework established by Implicit Diffusion Q-learning [17]. While the updates for the Q-function and value function remain consistent with IQL (see Equation D.2), IFQL distinguishes itself by employing a flow-matching objective for policy optimization, as defined by the behavior cloning loss in Equation 2.3.

The policy’s behavior cloning loss in DIFQL is formulated as:

$$\mathcal{L}_\pi^{\text{DIFQL}}(\theta) = \alpha \cdot \mathbb{E}_{a_1 \sim \mathcal{N}(0, I), t \sim \mathcal{U}[0, 1]} [\|v_\theta(a_t, t) - (a^{\text{target}} - a_1)\|^2], \quad a_t = (1 - t)a^{\text{target}} + ta_1 \quad (\text{E.2})$$

Actions are subsequently generated by sampling from the learned flow model (see Equation 2.4) and the final action selection is determined by maximizing the learned Q-value function (see Equation 2.8).

E.3 Efficient Implicit TrigFlow Q-Learning (ETrigFlow)

We present the Efficient TrigFlow (ETrigFlow) actor objective:

$$L_\pi^{\text{ETrigFlow}}(\theta) = \mathbb{E}_{(s, a_0, t, z) \sim \mathcal{D}} [-Q_\phi(s, f_\theta(a_t, t)) + \alpha \cdot \|f_\theta(a_t, t) - a_0\|_2^2] \quad (\text{E.3})$$

where $a_t = \cos(t) \cdot a_0 + \sin(t) \cdot z$ and $f_\theta(a_t, t) = \cos(t)a_t - \sin(t) \cdot F_\theta(a_t, t)$,

E.4 Directed Implicit TrigFlow Q-Learning (DTrigFlow)

For DTrigFlow, we build upon the policy diffusion formulation in Equation ?? and adopt the DOAL framework:

$$\mathcal{L}_\pi^{\text{DTrigFlow}}(\theta) = \mathbb{E}_{a_0 \sim p(a_0), z \sim \mathcal{N}(0, I), t \sim \mathcal{U}[0, \pi/2]} [\|f_\theta(a_t, t) - a_0^{\text{target}}\|_2^2] \quad (\text{E.4})$$

where $a_t = \cos(t)a^{\text{target}} + \sin(t)z$.

E.5 Directed MultiStep Flow Q-Learning (DMFQL)

The DMFQL share the same actor loss as the DIFQL, however it trains the value function through Q learning:

$$\mathcal{L}(\phi) = \mathbb{E}_{(\mathbf{s}, \mathbf{a}, r, \mathbf{s}', \mathbf{a}') \sim \mathcal{D}} [\|Q_\phi(\mathbf{s}, \mathbf{a}) - (r + \gamma (Q_{\phi'}(\mathbf{s}', \pi_\theta(\mathbf{s}'))))\|_2^2], \quad (\text{E.5})$$

where $\pi_\theta(\mathbf{s}')$ is sampled from MaxQ sampling with $n_{\text{target_sample}}$

E.6 Directed MultiStep Flow Regularized Q Learning (DMFReBRAC)

DMFReBRAC introduce an additional regularizer on the Q target.

$$\mathcal{L}(\phi) = \mathbb{E}_{(\mathbf{s}, \mathbf{a}, r, \mathbf{s}', \mathbf{a}') \sim \mathcal{D}} \left[\|Q_{\phi}(\mathbf{s}, \mathbf{a}) - (r + \gamma (Q_{\phi'}(\mathbf{s}', \pi_{\theta}(\mathbf{s}')) - \alpha_{\text{critic}} \cdot (\mathbf{a}' - \pi_{\theta}(\mathbf{s}'))^2))\|_2^2 \right], \quad (\text{E.6})$$

where $\pi_{\theta}(\mathbf{s}')$ is sampled from MaxQ sampling with $n_{\text{target_sample}}$ and α_{critic} controls the regularization.

Appendix F Experiment Results

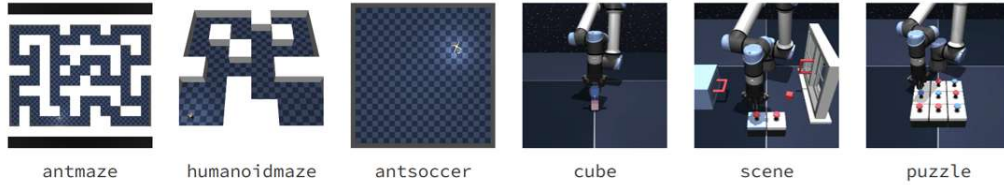


Figure F.1 Ogbench several environment visualizations

The figures [F.2](#) illustrate the performance curves of different algorithms across various environments. Specifically, each curve in the generated plots represents the performance trend achieved during the training process of the corresponding algorithm in a specific environment. The vertical axis displays the **normalized return**. This return is the direct evaluation result without applying any aggregation or averaging over multiple runs (e.g., without a moving average filter of size 3). The Antmaze Giant environment is deliberately excluded from this set of figures because the observed performance (normalized return) in these scenes was consistently near zero (≈ 0) for all algorithms tested, rendering the resulting plots visually uninformative.

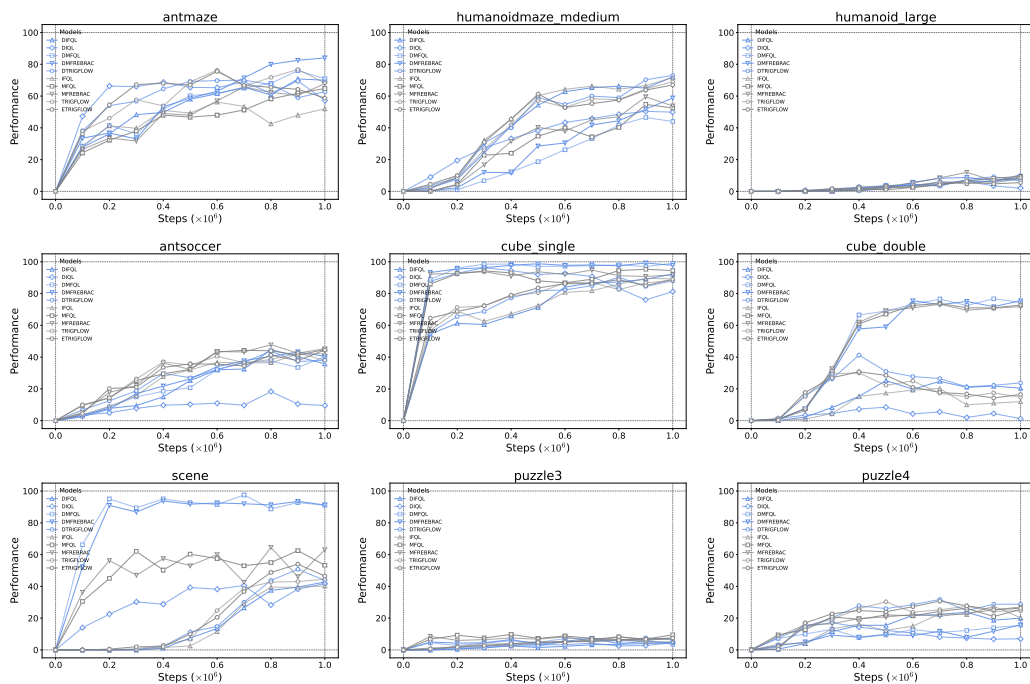


Figure F.2 Performance Comparison of Algorithms on the **OGbench** Environment.

Appendix G Ablation Study

In the following section, we present a detailed discussion on the influence of four critical hyperparameters: the max q sample n_{sample} , the brac coefficient α , the δ in DOAL, and the $n_{\text{target_sample}}$ for sample next action in DMFQL. Given the number of hyperparameters introduced in this work, and to rigorously demonstrate that our findings are not a consequence of aggressive "tuning", we adopt a strict and transparent two-stage evaluation strategy. Below, we detail the parameter search process and the measures taken to prevent overfitting and selection bias. All hyperparameter choices and initial ablation studies were conducted based on runs using four fixed random seeds: 11, 22, 33, and 44. Crucially, the final, definitive results presented in the main body of this paper are derived from a more robust set of eight independent random seeds: 111, 222, 333, 444, 555, 666, 777, and 888.

With IQL, we first choose n_{sample} on the Trigflow model, then choose δ on the DTrigflow model. Then, for all our models with IQL value function n_{sample} and δ are shared.

With Q-Learning, we choose n_{sample} on the MFQL model, and choose δ on the DMFQL model. On D4RL tasks, we copy the α_{critic} from ReBRAC and tune it on OGBench.

G.1 Choosing n_{sample}

Table G.1 demonstrates that a larger n_{sample} does not automatically translate into better performance. For instance, across the majority of environments, the optimal performance is achieved at relatively small n_{sample} values. In contrast, increasing n_{sample} to 64 or 128 often leads to a measurable decline in the average success rate. Our results demonstrate that MFQL benefits significantly from task-specific optimization of n_{sample} .

Table G.1 Comparison of MFQL Performance Across Different n_{sample} averaging over 4 seeds

Environment	n_{sample}							
	1	2	4	8	16	32	64	128
antmaze-large-navigate-singletask-v0	0.050	0.655	0.695	0.560	0.330	0.265	0.140	0.125
humanoidmaze-medium-navigate-singletask-v0	0.015	0.250	0.480	0.500	0.600	0.610	0.550	0.500
humanoidmaze-large-navigate-singletask-v0	0.000	0.005	0.045	0.075	0.095	0.045	0.040	0.035
antsoccer-arena-navigate-singletask-v0	0.005	0.165	0.355	0.500	0.460	0.400	0.295	0.285
cube-single-play-singletask-v0	0.105	0.965	0.950	0.870	0.780	0.675	0.645	0.575
cube-double-play-singletask-v0	0.005	0.290	0.700	0.625	0.455	0.350	0.215	0.135
scene-play-singletask-v0	0.040	0.550	0.665	0.635	0.545	0.630	0.520	0.550
puzzle-3x3-play-singletask-v0	0.005	0.090	0.040	0.000	0.000	0.000	0.000	0.000
puzzle-4x4-play-singletask-v0	0.000	0.045	0.205	0.095	0.080	0.055	0.065	0.050

G.2 Choosing δ

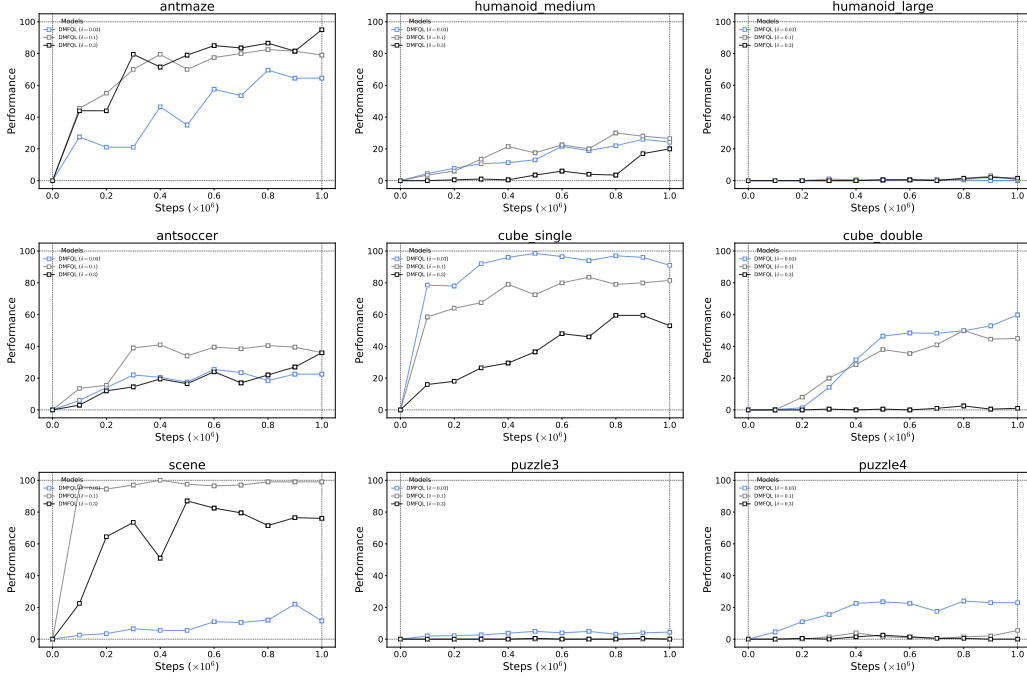


Figure G.1 Performance Comparison of **DMFQL** on the **OGBench** Environment with $\delta = 0.03, 0.1, 0.3$ over 4 seeds.

Regarding the δ hyperparameter, our analysis primarily serves to validate the effectiveness of DOAL. We strategically compared the performance of DMFQL using three distinct δ values: 0.03, 0.1, and 0.3. As demonstrated in the performance comparison (Figure G.1), the choice of δ has a major impact on the success rate across most environments. While we do not rule out the existence of even better δ values, our core objective was to show that the active learning mechanism introduced by the δ parameter is effective and necessary.

It should be noted that $\delta = 0$ recovers the baseline model exactly, but we do not consider this a valid hyperparameter choice.

G.3 Importance of Behavior Clone Coefficient α

We kept α from FQL [1] in the main experiments for controlling. In Figure G.2, on DMFQL, we compare the set α from FQL with $\alpha = 1$. We found that this parameter does not matter. In fact, one should realize α effectively controls the learning rate for actor networks and is no longer in charge of balancing in-distribution vs. Q value. With modern optimizers like ADAM [61], such explicit weighting is not needed.

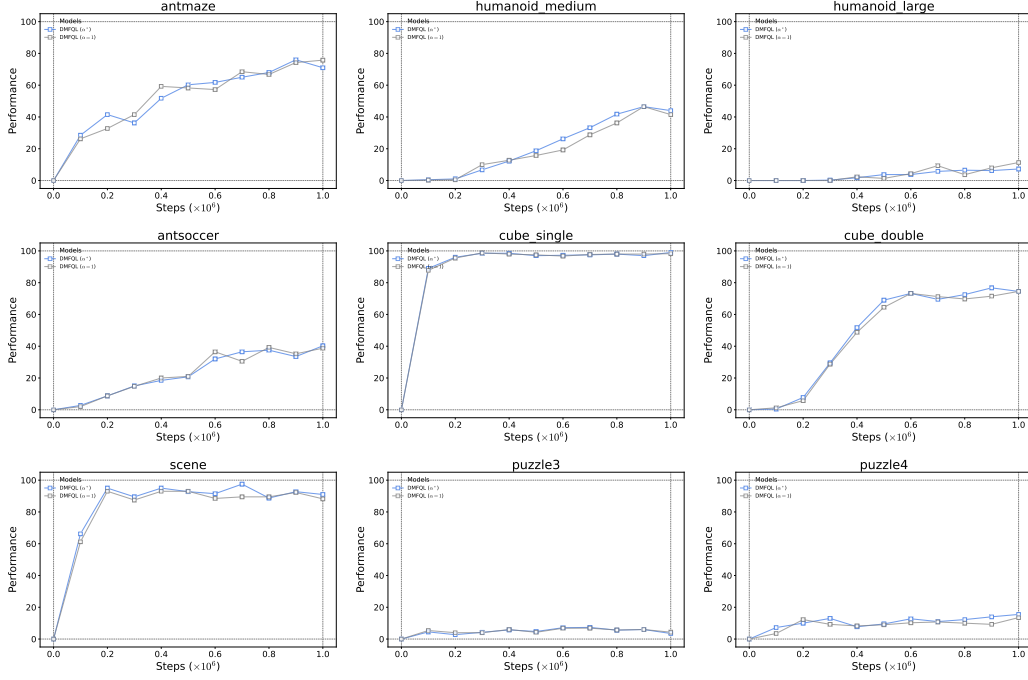


Figure G.2 Performance Comparison of DMFQL on the **OGBench** Environment with α^* from FQL and $\alpha = 1$ over 4 seeds.

G.4 Choosing $n_{\text{target_sample}}$

The $n_{\text{target_sample}}$ is necessary for computing the Bellman target value in the DMFQL Q-learning loss:

$$\mathcal{L}(\phi) = \mathbb{E}_{(\mathbf{s}, \mathbf{a}, r, \mathbf{s}', \mathbf{a}') \sim \mathcal{D}} \left[\|Q_{\phi}(\mathbf{s}, \mathbf{a}) - (r + \gamma (Q_{\phi'}(\mathbf{s}', \pi_{\theta}(\mathbf{s}'))))\|_2^2 \right],$$

where $\pi_{\theta}(\mathbf{s}')$ is an approximation of $\arg \max_{\mathbf{a}'} Q_{\phi'}(\mathbf{s}', \mathbf{a}')$. To estimate this maximum in a continuous action space, we employ MaxQ sampling by drawing $n_{\text{target_sample}}$ actions from the policy π_{θ} at state $\mathbf{s}'$ and taking the maximum Q-value. While MaxQ sampling can run samples in parallel, if the memory is full, increasing $n_{\text{target_sample}}$ will significantly increase the running time. We fix the $n_{\text{target_sample}} = 4$ for balancing computational budget and performance. In Figure G.3, we observe that $n_{\text{target_sample}} = 4$ could achieved better results than $n_{\text{target_sample}} = 1$ overall, but not always. We acknowledge that a more exhaustive hyperparameter search might yield slightly improved results for specific environments; our primary goal was to validate the effectiveness of the DOAL framework, not to achieve the absolute maximum score through extensive tuning.

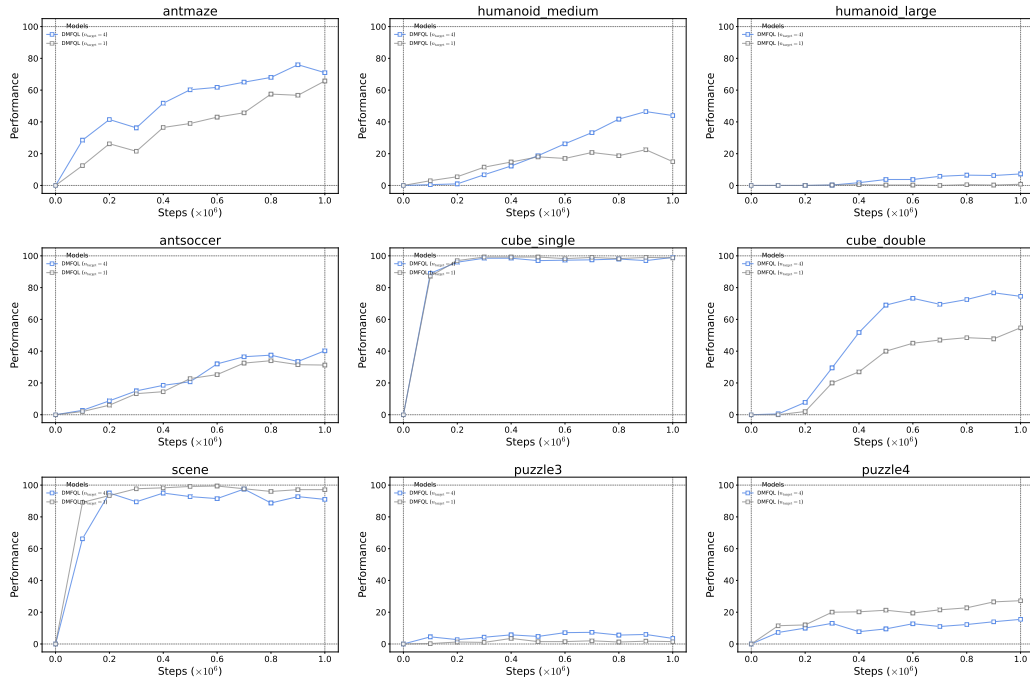


Figure G.3 Performance Comparison of **DMFQL** on the **OGBench** Environment with target n sample = 4 and target n sample = 1 over 4 seeds.

Appendix H Hyperparameters

Table H.1 DOAL Related Hyperparameters

Task	IQL-based		(Regularized) QL-based		
	δ	n_{sample}	δ	n_{sample}	α_{critic}
antmaze-large-navigate	0.1	4	0.03	4	0.01
humanoidmaze-medium-navigate	0.1	32	0.1	32	0.01
humanoidmaze-large-navigate	0.03	8	0.03	16	0.001
antsoccer-arena-navigate	0.1	16	0.1	16	0.1
cube-single-play	0.03	32	0.03	2	0.01
cube-double-play	0.1	16	0.03	4	0.1
scene-play	0.1	32	0.1	4	0.01
puzzle-3x3-play	0.03	4	0.03	2	0.01
puzzle-4x4-play	0.03	64	0.03	4	0.01
pen-human-v1	0.001	8	0.0003	4	0.5
pen-cloned-v1	0.001	16	0.0003	32	0.5
pen-expert-v1	0.001	64	0.0003	32	0.01
door-expert-v1	0.003	2	0.003	16	0.01
hammer-expert-v1	0.003	2	0.003	4	0.01
relocate-expert-v1	0.003	2	0.0003	4	0.01

Table H.1 summarizes the hyperparameters that we find. However, on D4RL tasks, we find that the variance is high across seeds. The optimal hyperparameters are not reliable. This might be another reason for the lack of improvement of DOAL models.

Table H.2 Other Hyperparameters

Hyperparameter	Value
Learning rate	0.0003
Optimizer	Adam [61]
Gradient steps	1000000 (OGBench), 500000 (D4RL)
Minibatch size	256
MLP dimensions	[512, 512, 512, 512]
Nonlinearity	GELU [62]
Target network smoothing coefficient	0.005
Discount factor γ	0.99
IQL expectile	0.9
Flow steps	10
Flow time sampling distribution	Unif([0, 1])